Universidad Mariano Gálvez de Guatemala

Sede El Naranjo

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Ing. Nancy Guadalupe Portillo Pinto



“Proyecto ACS”

Wendy Celeste Nathaly Estrada González

9490-22-3630

Aseguramiento de Calidad de Software

Guatemala 30 de Octubre del 2026

Índice

Introducción

Carta Presentación

1. Antecedentes del problema

1.1 Contexto Histórico del Transporte Público en el Área Metropolitana

Durante los años, el sistema de transporte público en la Ciudad de Guatemala y sus municipios aledaños se caracterizó por un modelo de operación desorganizado, fragmentado y con un esquema de gestión predominantemente empírico. El sistema tradicional, operado por múltiples asociaciones de transportistas privados, funcionaba bajo un esquema donde la competencia por el pasaje dictaba el comportamiento de las unidades en las calles. Esto generó un desorden vial crónico, donde las unidades se detenían en cualquier punto de la vía pública, obstaculizando el tránsito y poniendo en riesgo la integridad de los peatones y usuarios.

La falta de planificación tecnológica y urbana en este modelo tradicional resultó en una infraestructura deficiente. No existían estaciones controladas, los tiempos de espera eran incalculables debido a la ausencia de rutas estandarizadas y el manejo de los ingresos se realizaba en efectivo y sin ningún tipo de auditoría o control centralizado. Este escenario sentó las bases para una vulnerabilidad estructural que pronto sería aprovechada por estructuras criminales.

1.2. La Crisis de Inseguridad y el Colapso del Modelo Tradicional

El factor más crítico que impulsó la necesidad de un cambio radical en el paradigma de movilidad urbana fue la escalada de violencia y delincuencia que sufren muchos de los municipios de la capital. A partir de la década de los 2000, el transporte público tradicional se convirtió en uno de los sectores más golpeados por la criminalidad.

El manejo de grandes cantidades de dinero en efectivo a bordo de las unidades convirtió a los pilotos y ayudantes en blancos fáciles para el cobro de extorsiones por parte de pandillas y grupos del crimen organizado. Esta situación degeneró en un índice de robos alarmante, asaltos a mano armada a los pasajeros e innumerables atentados contra la vida de los conductores. La percepción de seguridad del usuario se desplomó, obligando a la población a buscar alternativas de movilidad más costosas y saturando aún más el parque vehicular de la ciudad. El colapso operativo y social de este sistema hizo evidente que cualquier nuevo modelo de transporte no solo debía resolver un problema de movilidad, sino principalmente un problema de seguridad pública.

1.3. El Surgimiento del Sistema Transmetro como Medida Alternativa

Ante el inminente colapso del transporte tradicional y la urgencia de salvaguardar la vida de los ciudadanos, las municipalidades optaron como medida alternativa el poder introducir el sistema Transmetro en toda la capital y municipios aledaños. Este modelo se basó en el concepto de Autobús de Tránsito Rápido (BRT, por sus siglas en inglés), el cual requería una reingeniería total de la infraestructura vial y del modelo de gestión.

El Transmetro se diseñó con un enfoque preventivo contra la criminalidad. Para lograr que no exista un índice de robos dentro del sistema, se implementaron cambios radicales:

Infraestructura Cerrada: Se construyeron paradas de bus establecidas y exclusivas, eliminando la posibilidad de que personas ajenas al sistema aborden las unidades en puntos no autorizados.

Eliminación del Efectivo a Bordo: La implementación de métodos de pago electrónicos en las estaciones eliminó el incentivo principal para los asaltos y extorsiones dentro de las unidades.

Profesionalización del Personal: A diferencia del sistema anterior, los pilotos del Transmetro son mejor reclutados, sometidos a pruebas psicológicas, historial de antecedentes y capacitaciones constantes, dignificando la labor del conductor.

1.4. El Desafío de la Expansión Metropolitana y la Descentralización

El éxito inicial del Transmetro en las arterias principales de la capital demostró que un transporte público seguro y ordenado era viable. Esto provocó un efecto dominó en el área metropolitana. Al notar la eficiencia del modelo, diversas municipalidades aledañas comenzaron a buscar la integración o replicación de este sistema para conectar a sus poblaciones pendulares (ciudadanos que residen en municipios periféricos, pero trabajan o estudian en la capital).

Sin embargo, esta expansión trajo consigo un nivel de complejidad operativa sin precedentes. A medida que se sumaron nuevas líneas, cientos de estaciones y una flota vehicular cada vez mayor, la administración manual o mediante sistemas informáticos básicos dejó de ser sostenible. La logística de coordinar flotas intermunicipales requiere rastrear con precisión métricas que antes no se consideraban críticas: distancias exactas entre paradas, asignación y cambio controlado de parqueos, y la gestión masiva de personal de seguridad y operación en múltiples jurisdicciones.

1.5. Brecha Tecnológica y la Necesidad de Aseguramiento de la Calidad (Software)

Es en este punto de inflexión donde se enmarca el presente proyecto. A pesar de los grandes avances en infraestructura física y seguridad perimetral, el sistema enfrenta actualmente una "brecha tecnológica" en su control interno. La recolección de información sobre las líneas de transmetro de la ciudad de Guatemala y sus municipios requiere de una arquitectura de software robusta y altamente escalable.

Actualmente, el manejo de las reglas de negocio (cómo evitar que un bus se asigne a múltiples líneas o asegurar que cada estación cuente con al menos un guardia por acceso) representa un reto enorme si no está respaldado por bases de datos relacionales íntegras. Más aún, la dinámica moderna del transporte exige la toma de decisiones en tiempo real basada en análisis de datos, como la capacidad de generar alertas automáticas cuando una estación supera su capacidad máxima o aplicar penalizaciones de tiempo a unidades vacías.

La falta de una red interconectada entre las computadoras de las diferentes estaciones es un obstáculo crítico que impide la trazabilidad de los métodos de pago y el monitoreo en vivo de la flota. Por lo tanto, el desarrollo de un sistema de control de Aseguramiento de la Calidad del Software (SQA) no es solo una mejora administrativa, sino un pilar fundamental para garantizar que el índice de delincuencia cero se mantenga, que los recursos municipales se optimicen y que el nivel de servicio al usuario justifique la inversión pública a largo plazo.

2. Definición de la situación actual

2.1. Contexto General y Despliegue Operativo

En la actualidad, las municipalidades de la capital y sus municipios aledaños han optado por introducir el transmetro como una medida alternativa para contrarrestar la delincuencia. Este sistema se encuentra en una fase operativa donde ha logrado consolidar aspectos clave para la seguridad ciudadana: se han establecido paradas de bus físicas, se cuenta con pilotos que son mejor reclutados y se mantiene un escenario donde no hay un índice de robos.

Sin embargo, desde el punto de vista del Aseguramiento de la Calidad del Software (SQA) y la gestión de la información, el escenario actual presenta una alta complejidad logística que demanda un sistema de control centralizado. Se requiere recoger y estructurar de manera sistemática toda la información operativa de las líneas de transmetro que atraviesan la ciudad de Guatemala y sus municipios. En el momento presente, la coordinación de esta información se divide en múltiples frentes (infraestructura, flota, personal y reglas operativas) que deben ser mapeados con precisión para la propuesta del nuevo sistema de control.

2.2. Arquitectura de Rutas, Estaciones y Jurisdicciones

La red física del sistema de transporte se compone actualmente de una estructura jerárquica y relacional que define la movilidad de las unidades. A nivel jurisdiccional, la situación actual estipula que cada línea y estación que existe pertenece a una municipalidad específica. Esto implica que el sistema opera en un entorno intermunicipal que requiere un control detallado de la ubicación de sus activos.

En cuanto a la configuración de las rutas, la situación operativa define que una línea está compuesta por una serie de estaciones dispuestas en un orden determinado. Es un requerimiento absoluto para la operación actual recoger la información de este orden, abarcando tanto el diseño de la ruta como las visitas secuenciales que cada unidad de transporte vaya a realizar durante el momento de su recorrido.

La interacción entre líneas y estaciones presenta una topología de red compleja:

Cada estación pertenece al menos a una línea, pero la operación permite que una estación pueda pertenecer a varias líneas simultáneamente (puntos de transbordo). Para la planificación espacial y de tiempos, actualmente se necesita saber qué distancia existe de una estación a otra. Asimismo, como métrica global de operación, se necesita saber qué distancia total tiene una línea completa.

2.3. Control de Accesos y Seguridad Física en Estaciones

La infraestructura de las estaciones ha sido diseñada para controlar el flujo de pasajeros de forma segura y estructurada, lo cual impacta directamente en la necesidad de control del sistema informático actual. Físicamente, la arquitectura establece que cada estación puede tener varios accesos, mientras que un acceso sólo puede pertenecer a una estación en particular. Desde el punto de vista operativo y de monitoreo, al sistema le interesa conocer todos los accesos específicos de cada línea para mapear los puntos de entrada de los usuarios.

La seguridad, siendo el pilar fundamental que motivó este modelo de transporte, impone restricciones estrictas sobre el personal desplegado. En la situación actual, las normativas exigen que las estaciones cuenten por lo menos con un guardia de seguridad asignado por cada acceso disponible. Esto genera una matriz de asignación de recursos humanos que debe ser controlada continuamente para garantizar la operatividad de cada parada de bus.

2.4. Gestión de los Buses e Infraestructura de Parqueo

La administración de los activos móviles (buses) y estáticos (parqueos) es uno de los componentes más críticos y estrictos de la situación actual. La asignación de la flota está regida por reglas matemáticas y lógicas que buscan optimizar el uso de los vehículos sin saturar las rutas ni dejar estaciones desatendidas:

Cada línea tiene asignados una serie de buses de forma exclusiva.

La regla de negocio actual impide que suceda que un bus esté asignado a más de una línea al mismo tiempo.

Se contempla la existencia de una flota de reserva, por lo que es permitido que un bus no esté asignado a ninguna línea temporalmente.

Existe un balance de carga por ruta: cada línea tiene asignados como mínimo tantos buses como estaciones tenga, y como máximo el doble del número de estaciones.

Por otro lado, la logística nocturna y de resguardo de las unidades exige un control riguroso de la infraestructura de parqueos. En la actualidad, solo algunas estaciones tienen asignados parqueos en sus instalaciones. Sin embargo, la regla operativa obliga a que cada bus tenga asignado un parqueo en todo momento. La flexibilidad del sistema permite que un bus pueda cambiar de parqueo según las necesidades operativas, pero bajo ninguna circunstancia puede quedarse sin él.

2.5. Dinámica de Aforos, Tiempos de Espera y Reglas de Negocio en Tiempo Real

El control del volumen de pasajeros es el mayor desafío operativo en la situación actual. Los buses que conforman la flotilla tienen una capacidad máxima de pasajeros establecida por el fabricante y las normativas de tránsito. Para mantener la calidad del servicio y evitar el hacinamiento en los andenes, el sistema exige mecanismos de respuesta inmediata basados en el flujo de usuarios:

Alerta de Saturación: en el momento en que la cantidad de pasajeros esperando en una estación supera por lo menos un 50 % más de su capacidad, es decir, el 150 % del aforo máximo de la estación, la operación exige generar una alerta para que otro bus llegue a solventar la demanda.

Penalización por Baja Demanda: Por el contrario, para evitar el desperdicio de combustible y optimizar las frecuencias, si un bus no llena el 25% de su capacidad al llegar, la regla estipula que deberá esperar 5 minutos más en cada estación para recolectar más usuarios antes de continuar su recorrido.

Actualmente, procesar estas variables matemáticas de aforo y tiempo de espera en tiempo real representa una necesidad urgente que justifica la implementación de la nueva plataforma de software.

2.6. Estado de la Infraestructura Tecnológica y Telecomunicaciones

A nivel de hardware y redes, la situación actual se encuentra en una fase de diseño y provisión de equipo. Es importante mencionar que la planificación estratégica espera tener una PC de trabajo instalada para cada estación de bus. Esta infraestructura de hardware requiere incluir a un operador designado que pueda tener el control de dichas máquinas en el sitio.

A pesar de contar con la disposición de adquirir el hardware necesario, el proyecto enfrenta un vacío a nivel de arquitectura de redes. Al día de hoy, aún no se ha determinado cuál es la mejor forma tecnológica o de telecomunicaciones para conectar las máquinas de un lugar a otro (LAN, WAN, VPN, conexiones en la nube, etc.). Esta indefinición técnica afecta la forma en que los datos fluyen entre las estaciones locales y el servidor centralizado de la propuesta.

Finalmente, en cuanto al modelo de recaudación financiera y acceso de los usuarios, la situación operativa ha establecido como estándar que los métodos de pago se realicen exclusivamente a través de una tarjeta de acceso. La integración de los lectores de estas tarjetas en las estaciones con las computadoras de los operadores será fundamental para registrar los ingresos y el flujo de pasajeros que dispara las alertas de capacidad mencionadas anteriormente.

2.7. Registro y Control del Capital Humano Central

Más allá del personal de campo y seguridad, el soporte administrativo requiere llevar un control estricto de los conductores que tienen a su cargo las unidades. La situación actual exige que para los pilotos se necesita conocer detalladamente su historial educativo, sus datos de residencia exacta y sus canales de comunicación. Esta recopilación de datos no solo responde a políticas de recursos humanos, sino que es una extensión del estricto filtro de seguridad que caracteriza al reclutamiento del Transmetro, asegurando la trazabilidad total de los perfiles de quienes operan el sistema.

3. Deficiencias de los controles actuales

Carencia de Integración Tecnológica Centralizada

El principal vacío operativo radica en la ausencia de una arquitectura de red definida, ya que aún no se ha determinado cuál es la mejor forma de conectar las máquinas de un lugar con otro. Esto impide la consolidación de datos transaccionales en un Data Warehouse o entorno centralizado, limitando drásticamente el control que el operador local puede ejercer desde la PC asignada en su estación de bus. Sin una red robusta, la comunicación de los sistemas del área metropolitana de Guatemala queda aislada.

3.1. Vulnerabilidades en el Control de Buses y Asignación

La gestión actual del transporte genera riesgos críticos en la integridad referencial de los recursos físicos, requiriendo validaciones estrictas a nivel de base de datos:

Al carecer de bloqueos sistémicos, existe el riesgo de que un bus quede asignado a más de una línea simultáneamente. No existe un mecanismo (como triggers o procedimientos

almacenados) que garantice el balance de carga: asignar a cada línea como mínimo tantos buses como estaciones tenga, y como máximo el doble de su número de estaciones.

El control de infraestructura es vulnerable a errores humanos, permitiendo el riesgo de que un bus pueda quedarse sin un parqueo asignado.

3.2. Deficiencias en el Monitoreo de Aforo y Tiempos

La operación carece de inteligencia de negocios en tiempo real para gestionar el flujo de pasajeros y optimizar las frecuencias de las rutas:

Falta un sistema de monitoreo automatizado que genere una alerta inmediata para despachar otro bus en el momento que una estación supera por lo menos un 50% más de su capacidad. No hay controles lógicos para obligar a una unidad a esperar 5 minutos más en cada estación si el bus no llena el 25% de su capacidad.

La ausencia de un sistema que calcule automáticamente qué distancia existe de una estación a otra, o qué distancia total tiene una línea, dificulta la planificación logística.

3.3. Riesgos en la Seguridad de Datos y Personal

El modelo adolece de controles para la auditoría continua del factor humano y las transacciones financieras:

El ecosistema de cobro mediante métodos de pago con tarjeta de acceso requiere un entorno seguro para evitar fraudes en el uso del servicio. Existe un control deficiente sobre los perfiles de los pilotos, dificultando conocer al instante su historial educativo, datos de residencia y medios de comunicación.

El sistema no audita de forma proactiva que se cumpla la restricción de mantener por lo menos un guardia de seguridad por cada acceso de la estación.

4. Planificación

5. Análisis de Requerimientos

Objetivo del sistema

El objetivo principal del sistema será proporcionar una herramienta tecnológica integral para la administración y supervisión del sistema Transmetro, permitiendo controlar los principales recursos involucrados en la operación y automatizar las reglas de negocio establecidas.
La plataforma permitirá centralizar la información de las diferentes estaciones y líneas, controlar la asignación de buses y parqueos, administrar el personal involucrado en la operación y seguridad, monitorear la demanda de pasajeros y generar alertas cuando las condiciones de operación lo requieran.
De esta manera, la solución pretende convertirse en un punto central de control y consulta para la gestión del transporte, proporcionando información confiable que permita actuar oportunamente ante situaciones que puedan afectar la calidad del servicio

Requerimientos funcionales

RF-01. Gestión de Municipalidades

El sistema deberá permitir administrar las municipalidades en las que opera el servicio de transporte, manteniendo centralizada la información necesaria para relacionarlas con las líneas y estaciones correspondientes.

El módulo deberá permitir:

- Crear registros de municipalidades indicando código, nombre y estado.
- Actualizar la información de una municipalidad.
- Consultar las municipalidades disponibles y su estado operativo.
- Asociar las líneas y estaciones correspondientes a una municipalidad.
- Impedir que una estación sea asociada a más de una municipalidad.

cada estación deberá pertenecer únicamente a un municipio, aunque un mismo municipio podrá contar con múltiples estaciones.

RF-02. Gestión de Líneas y Recorridos

El sistema deberá permitir administrar las líneas de transporte y definir el recorrido que realizará cada una mediante la asociación ordenada de estaciones.

El módulo deberá permitir:

- Crear una línea indicando código, nombre, municipalidad, tipo de servicio (paradas continuas o directo), tarifa, color, si la ruta es circular y estado.
- Asociar estaciones a una línea y definir el orden en que serán recorridas.
- Registrar la distancia desde la parada anterior para cada estación del recorrido.
- Calcular automáticamente la distancia total de la línea como la suma de sus tramos (trigger trg_linea_estacion_distancia). Este dato no se ingresa a mano.
- Modificar el recorrido manteniendo un orden consecutivo.
- Consultar el recorrido completo, las estaciones y todos los accesos de la línea (punto 8 del enunciado).

Una misma estación podrá formar parte de diferentes líneas cuando los recorridos la compartan; por ejemplo, San Sebastián es estación de las Líneas 1 y 2.

RF-03. Gestión de Estaciones

El sistema deberá proporcionar un módulo para administrar las estaciones que forman parte de la infraestructura del servicio de transporte.

Cada estación deberá contar con la siguiente información:

- Identificador.
- Municipalidad a la que pertenece.
- Nombre y dirección.
- Aforo máximo y aforo actual.
- Coordenadas geográficas (latitud y longitud).

El sistema deberá permitir:

- Crear y actualizar estaciones.
- Asociar una estación con una o varias líneas.
- Controlar la capacidad máxima y la ocupación actual (porcentaje de aforo).
- Ubicar la estación en el mapa de rutas.
- Relacionar sus accesos, molinetes, guardias y parqueos.

La situación de la estación se conoce por su aforo y por sus alertas abiertas; no se maneja un campo de “estado operativo” independiente.

RF-04. Gestión de Accesos y Molinetes

El sistema deberá permitir administrar los accesos físicos disponibles en cada estación y los molinetes asociados a dichos accesos.

El módulo deberá permitir:

- Crear accesos asociados a una estación, indicando su ubicación física (por ejemplo, “Acceso Norte”).
- Asociar un molinete propio a cada acceso, con su dirección IP.
- Registrar el estado operativo del molinete: activo, inactivo o en mantenimiento.
- Asignar personal de seguridad a cada acceso.
- Consultar todos los accesos de una estación y de una línea.

Reglas de negocio:

- Un acceso pertenece a una sola estación (punto 3).
- Cada estación deberá contar con al menos dos accesos físicos. Es una regla definida por el equipo; el trigger trg_accesos_min_dos_before_delete impide eliminar el penúltimo acceso.
- Cada acceso dispone de un molinete propio (relación 1 a 1).
- Cada acceso deberá contar con al menos un guardia de seguridad (punto 10, trigger trg_seguridad_min_uno).

RF-05. Gestión Integral de la Flota

El sistema deberá permitir administrar las unidades de transporte que conforman la flota y controlar su asignación a las líneas activas.

Para cada bus se deberá manejar:

- Identificador.
- Placa.
- Capacidad máxima.
- Línea asignada.
- Estado operativo.

El sistema deberá permitir:

- Incorporar nuevas unidades.
- Actualizar la información de los buses.
- Consultar el estado de cada unidad.
- Asociar un bus a una línea activa.
- Retirar o desactivar temporalmente una unidad.
- Identificar los buses actualmente asignados a cada línea.

una unidad de transporte solamente podrá estar asignada a una línea activa a la vez.

RF-06. Gestión de Parqueos

El sistema deberá administrar los parqueos con los que cuentan algunas estaciones y garantizar que cada bus tenga siempre un parqueo asignado (puntos 6 y 7 del enunciado).

El sistema deberá permitir:

- Registrar los parqueos (bahías) de una estación con un código único.
- Asignar un parqueo a cada bus. Un parqueo solo puede tener un bus y un bus solo ocupa un parqueo.
- Cambiar el parqueo de un bus mediante el procedimiento almacenado sp_cambiar_parqueo_bus, que libera el parqueo anterior y ocupa el nuevo en una sola transacción.
- Consultar los parqueos libres y ocupados de cada estación.

Regla de negocio: un bus puede cambiar de parqueo, pero nunca quedarse sin él. La asignación es fija hasta que el administrador la cambia; no es una asignación dinámica por orden de llegada.

RF-07. Gestión del Personal Operativo

El sistema deberá permitir administrar el personal que participa en la operación del servicio de transporte, diferenciando las funciones de **Piloto, Seguridad y Operador**.

Para todos los empleados se deberá gestionar:

- Nombre completo.
- Fecha de nacimiento.
- DPI.
- Número de teléfono.
- Dirección residencial.
- Estado del empleado.

Adicionalmente, el sistema deberá manejar información específica según el rol.

Pilotos

- Número de licencia.
- Tipo de licencia.
- Años de experiencia.
- Nivel educativo.

Personal de seguridad

- Permisos vigentes.
- Constancia de antecedentes penales.
- Número de placa.

Operadores

- Nivel de estudios.
- Años de experiencia en monitoreo y control.

Estas características corresponden a la diferenciación de roles definida en el documento del proyecto.

RF-08. Gestión de Asignación de Personal

El sistema deberá permitir relacionar al personal operativo con los recursos que tiene bajo su responsabilidad.

El módulo deberá permitir:

- Asignar pilotos a las unidades correspondientes.
- Asociar personal de seguridad con los accesos de cada estación.
- Asignar operadores a la PC de trabajo de una estación (punto 16).
- Consultar el personal actualmente asignado.
- Impedir que un acceso quede sin guardia: el sistema rechaza eliminar o reasignar el último guardia de un acceso y muestra un mensaje claro.

Todo acceso deberá contar como mínimo con un elemento de seguridad responsable (punto 10).
El control es preventivo: la operación se bloquea en lugar de solo generar una alerta.

RF-09. Monitoreo Operativo de Estaciones

El sistema deberá proporcionar a los operadores una interfaz centralizada para supervisar el funcionamiento de su estación.

Desde la PC de trabajo de su estación, el operador deberá poder:

- Consultar el aforo actual y el porcentaje de ocupación de la estación.
- Actualizar la cantidad de pasajeros en espera.
- Consultar los buses de las líneas que pasan por la estación, su ocupación y si deben esperar 5 minutos extra.

- Consultar el horario de servicio y el mapa de rutas.
- Dar seguimiento y atender las alertas de refuerzo de la estación.
- Registrar los pases en los molinetes de la estación.

La ubicación de los buses en tiempo real (GPS) y el control de bahías de abordaje quedan fuera del alcance de esta versión y se proponen para una fase 2.

RF-10. Gestión de Pasajeros y Ocupación

El sistema deberá permitir registrar y controlar la cantidad de pasajeros presentes en las estaciones y en las unidades de transporte para determinar el nivel de ocupación y apoyar la toma de decisiones operativas.

El sistema deberá:

- Registrar la cantidad de pasajeros esperando en cada estación (aforo actual).
- Calcular el porcentaje de ocupación de la estación respecto de su aforo máximo.
- Registrar el porcentaje de ocupación de cada bus.
- Identificar situaciones de alta acumulación de pasajeros.
- Usar los niveles de ocupación como condición para generar acciones operativas.

Regla de negocio (punto 14): cuando una estación tenga por lo menos un 50 % más de su capacidad, es decir, cuando el aforo actual sea mayor o igual al 150 % del aforo máximo, el sistema generará automáticamente una alerta para que otro bus llegue (trigger trg_estaciones_alerta_aforo). Ejemplo: una estación con aforo de 300 personas genera la alerta al llegar a 450.

RF-11. Gestión de Buses de Refuerzo y Límite de Flota

El sistema deberá apoyar la incorporación de unidades adicionales cuando la demanda lo requiera, respetando el tamaño de flota permitido para cada línea.

Cuando se genere una alerta de refuerzo, el sistema deberá:

- Registrar la alerta con la estación, la descripción y la fecha y hora.
- Mostrarla en el panel del operador y del administrador.
- Permitir que el operador la atienda (solicitud del bus) y registrar la fecha de cierre.
- Permitir asignar un bus disponible a la línea, siempre que no se supere el máximo de flota.

Regla de negocio (punto 5): cada línea debe tener como mínimo tantos buses como estaciones (n) y como máximo el doble (2n). El trigger trg_buses_max_flota rechaza cualquier asignación que supere 2n, y el reporte de líneas señala las que no alcanzan el mínimo.

RF-12. Gestión de Tiempos de Espera

El sistema deberá calcular y controlar los tiempos de permanencia de las unidades en las estaciones de acuerdo con su nivel de ocupación.

El sistema deberá:

- Registrar la hora de llegada y de salida de un bus en cada estación.
- Calcular el tiempo de permanencia.
- Registrar el porcentaje de ocupación del bus al llegar.
- Identificar unidades con baja ocupación.
- Indicar al operador los minutos de espera adicionales que corresponden.

Regla de negocio (punto 15): si un bus no llena el 25 % de su capacidad, deberá esperar 5 minutos más en cada estación, además de su tiempo normal de parada.

RF-13. Gestión de Viajes y Seguimiento de Recorridos

El sistema deberá permitir controlar los viajes realizados por cada unidad dentro de una línea determinada.

Cada viaje deberá registrar:

- Bus utilizado.
- Línea correspondiente.
- Fecha y hora de inicio.
- Fecha y hora de finalización.
- Estado del viaje.

Durante el recorrido, el sistema podrá asociar las visitas realizadas a las diferentes estaciones, registrando:

- Estación visitada.
- Orden de visita.
- Hora de llegada.
- Hora de salida.
- Nivel de ocupación registrado.

Esto permitirá construir el historial operativo de cada unidad y determinar cómo se desarrolló un recorrido.

RF-14. Métodos de Pago y Transacciones en Molinetes

El sistema deberá administrar las transacciones realizadas en los molinetes de las estaciones. De acuerdo con la operación actual del Transmetro, el molinete no acepta efectivo.

El sistema deberá permitir procesar:

- Tarjeta Ciudadana (tarjeta de acceso oficial): se descuentan primero los viajes gratis y después el saldo.
- Tarjeta bancaria de débito o crédito sin contacto.

Cada transacción deberá almacenar como mínimo:

- Identificador de la transacción.
- Molinete utilizado.
- Línea y tarifa aplicada.
- Tarjeta utilizada, cuando es Tarjeta Ciudadana.

- Monto y si fue un viaje gratis.
- Método de pago.
- Fecha y hora.

Reglas de negocio: no se registra un pase si la tarjeta está bloqueada o no tiene saldo suficiente, y el monto debe ser mayor a cero salvo en los viajes gratis. La emisión y recarga de la Tarjeta Ciudadana se describen en el RF-21.

RF-15. Gestión de Alertas Operativas

El sistema deberá generar y administrar alertas relacionadas con situaciones que puedan afectar la continuidad o eficiencia del servicio.

Tipos de alerta contemplados en la base de datos:

- refuerzo_bus: alta acumulación de pasajeros. Se genera automáticamente.
- espera_minima: bus con ocupación menor al 25 %.
- bahia_ocupada: reservada para el control de bahías de la fase 2.
- otro: situaciones relacionadas con la operación de la estación.

Cada alerta deberá registrar:

- Tipo de alerta.
- Estación relacionada.
- Descripción.
- Fecha y hora de creación.
- Estado: abierta, atendida o cerrada.
- Fecha de cierre.

El usuario que atiende la alerta queda registrado en la bitácora de auditoría. La falta de guardia y el exceso de flota no generan alerta porque el sistema los impide directamente (RF-08 y RF-11).

El sistema deberá disponer de un panel donde operadores y administradores puedan visualizar y dar seguimiento a las alertas pendientes.

RF-16. Gestión de Usuarios, Roles y Permisos

El sistema deberá proporcionar un mecanismo de administración de usuarios que permita controlar el acceso a las diferentes funcionalidades según las responsabilidades asignadas.

Los usuarios deberán contar con:

- Credenciales de acceso (correo y contraseña cifrada).
- Rol dentro del sistema.
- Empleado vinculado, cuando corresponda (por ejemplo, el operador y su estación).

El sistema deberá controlar que cada usuario únicamente pueda ejecutar las operaciones correspondientes a su rol (middleware VerificarRol en cada ruta).

Roles implementados:

- **Administrador:** configuración y administración general, tarjetas y reportes.

- **Operador:** monitoreo y operación de la estación asignada.

Roles propuestos para la fase 2:

- **Piloto:** consulta del recorrido y de las paradas asignadas.
- **Seguridad:** registro de incidencias en los accesos.

RF-17. Gestión de Auditoría

El sistema deberá registrar las operaciones relevantes realizadas por los usuarios para garantizar la trazabilidad de las acciones ejecutadas dentro de la plataforma.

La bitácora deberá permitir almacenar:

- Usuario que realizó la operación.
- Tabla o módulo afectado.
- Registro afectado.
- Tipo de acción (INSERT, UPDATE o DELETE).
- Datos anteriores, cuando corresponda.
- Datos posteriores.
- Fecha y hora.

El administrador deberá poder consultar la bitácora para determinar quién realizó una modificación y cuándo.

Estado actual: la tabla bitacora_auditoria ya registra los cambios del panel del operador y del trazo de las líneas. Falta extender el registro a los módulos administrativos y construir la pantalla de consulta.

RF-18. Comunicación entre Estaciones y Servicios Centrales

El sistema deberá permitir que el personal autorizado de cada estación acceda a la plataforma mediante Internet desde el equipo asignado a la estación.

La aplicación deberá funcionar mediante el siguiente flujo:

Computadora de estación (navegador) → HTTPS → Aplicación Laravel en Render → Base de datos PostgreSQL en Supabase

Las acciones realizadas desde una estación deberán ser enviadas al backend para:

1. Validar al usuario.
2. Verificar sus permisos.
3. Validar la información recibida.
4. Aplicar las reglas de negocio.
5. Registrar o actualizar la información.
6. Generar las alertas correspondientes.
7. Devolver el resultado de la operación al usuario.

No se permitirá que los equipos de las estaciones establezcan una conexión directa con la base de datos central.

En caso de pérdida de conexión a Internet, la operación no se registra a medias: el sistema informa al operador que no pudo procesarse y la operación puede repetirse cuando se restablezca la comunicación. Un modo sin conexión con almacenamiento local queda como mejora futura.

RF-19. Gestión de Reportes Operativos

El sistema deberá generar reportes que apoyen la supervisión y administración del servicio (punto 15 del enunciado). Los reportes se muestran en la aplicación web, se pueden imprimir o exportar a CSV y cada uno muestra la consulta SQL utilizada, para dejar constancia de ella en este documento.

Reportes implementados:

- Estaciones: municipalidad, líneas, accesos, guardias, parqueos y aforo.
- Líneas: estaciones, distancia, flota mínima, máxima y asignada.
- Buses asignados por línea.
- Recorridos: viajes realizados.
- Visitas y ocupación por estación.

Reportes pendientes:

- Alertas generadas y atendidas.
- Transacciones y recaudación por método de pago.
- Personal asignado.

Los reportes pueden filtrarse por línea y por municipalidad.

RF-20. Mapa de Rutas y Trazo de Líneas

El sistema deberá mostrar en un mapa las líneas del Transmetro con sus estaciones, utilizando información oficial de la Municipalidad de Guatemala.

El sistema deberá permitir:

- Visualizar todas las líneas con su color oficial y sus estaciones sobre un mapa de OpenStreetMap.
- Importar el trazo oficial de una línea desde un archivo KML o KMZ.
- Editar el trazo manualmente y ajustarlo a las calles.
- Ordenar las estaciones según su posición sobre el trazo y calcular la distancia entre ellas.
- Mostrar en cada estación su nombre, dirección y las líneas que la atienden.

RF-21. Tarjeta Ciudadana y Recargas

El sistema deberá administrar la tarjeta de acceso indicada en el punto 18 del enunciado.

El sistema deberá permitir:

- Emitir tarjetas de tipo ciudadana, adulto mayor o persona con discapacidad. Los dos últimos tipos requieren el nombre y el DPI del titular.
- Registrar el costo de emisión de Q20.00, que incluye 5 viajes gratis.

- Registrar recargas de Q5.00 a Q200.00 en efectivo, débito, crédito o en línea, en el punto de recarga (no en el molinete).
- Consultar el saldo, los viajes gratis restantes y el historial de recargas y pases.
- Bloquear o reactivar una tarjeta.

RF-22. Horarios de Servicio

El sistema deberá registrar el horario de servicio de cada línea para tres tipos de día: lunes a viernes, sábado, y domingo y feriados, con hora de inicio y de fin tomadas de la información oficial. El sistema deberá indicar si una línea está en servicio en un momento dado.

Matriz de trazabilidad de requerimientos funcionales

La matriz relaciona cada requerimiento con el punto del enunciado que atiende y con la evidencia en el sistema construido, para verificar la cobertura del alcance.

RF	Punto del enunciado	Estado	Evidencia en el sistema
RF-01	11	Implementado	Módulo de municipios; FK en líneas y estaciones
RF-02	1, 2, 8, 12, 13	Implementado	linea_estacion; trg_linea_estacion_distancia; LineasOficialesTest
RF-03	2, 11, 14	Implementado	Admin/EstacionController; PantallasTest
RF-04	3, 8, 10	Implementado	trg_accesos_min_dos_before_delete; ReglasBaseDatosTest
RF-05	4	Implementado	BusController; restricción de una línea por bus
RF-06	6, 7	Implementado	sp_cambiar_parqueo_bus; UNIQUE en id_bus_asignado
RF-07	9	Implementado	PersonalController; datos de contacto cifrados
RF-08	10, 16	Implementado	trg_seguridad_min_uno; operador ligado a estación
RF-09	16	Parcial (sin GPS ni bahías)	Operador/EstacionController; OperadorTest
RF-10	14	Implementado	trg_estaciones_alerta_aforo (150 %)
RF-11	5, 14	Implementado	trg_buses_max_flota; reporte de líneas
RF-12	15	Implementado (cálculo)	Bus::minutosEsperaExtra(); ReglasModelosTest
RF-13	1	Parcial (sin pantalla de captura)	Tablas viajes y visitas_viaje; reporte de recorridos
RF-14	18	Implementado	Servicio CobroPasaje; CobroPasajeTest
RF-15	14	Parcial (solo refuerzo)	alertas_sistema; panel del operador
RF-16	—	Parcial (2 de 4 roles)	Middleware VerificarRol; AutenticacionTest
RF-17	—	Parcial (sin pantalla)	Tabla bitacora_auditoria
RF-18	16, 17	Implementado	HTTPS; sin conexión directa a la base
RF-19	15	Parcial (5 de 8 reportes)	ReporteController; ReportesTest
RF-20	1, 12	Implementado	ImportadorKml; ImportarKmlTest; TrazoRutaTest
RF-21	18	Implementado	TarjetaController; TarjetasTest
RF-22	—	Implementado	horarios_linea; LineasOficialesTest

Requerimientos No Funcionales

Los requerimientos no funcionales establecen las condiciones de calidad y las características que deberá cumplir el Sistema de Control de Transmetro para garantizar que la solución sea segura, confiable, eficiente y adecuada para las necesidades operativas de las municipalidades.
Estos requerimientos no describen una función específica del sistema, sino las condiciones bajo las cuales dichas funciones deberán operar correctamente.

RNF-01. Seguridad

La aplicación deberá implementar mecanismos de autenticación y autorización que permitan proteger la información almacenada y procesada por el sistema.
Cada usuario deberá contar con credenciales propias para acceder a la plataforma. El sistema deberá identificar al usuario y validar que tenga autorización para realizar la operación solicitada.
El acceso a las funcionalidades deberá establecerse de acuerdo con el rol asignado al usuario. Por ejemplo, un administrador podrá gestionar líneas, estaciones, buses y usuarios, mientras que un operador de estación tendrá acceso únicamente a las funciones necesarias para realizar sus actividades.
Asimismo, las operaciones relacionadas con información sensible, como datos de pilotos, personal de seguridad, usuarios y transacciones de tarjetas de acceso, deberán estar restringidas a usuarios autorizados.
El objetivo es poder reducir el riesgo de accesos no autorizados, modificación indebida de información o exposición de datos del sistema.
El criterio de cumplimiento es que ningún usuario deberá poder acceder a módulos o información que no correspondan a los permisos establecidos para su rol.
Criterio medible: el 100 % de las rutas exige sesión y rol (verificado con AutenticacionTest y PantallasTest); contraseñas con bcrypt; protección CSRF en todos los formularios; datos sensibles del personal cifrados; Row Level Security activado en las 21 tablas.

RNF-02. Integridad de información

La base de datos deberá garantizar que la información relacionada con líneas, estaciones, buses, parqueos, pilotos y demás elementos del sistema mantenga relaciones consistentes.
La aplicación deberá validar las operaciones antes de almacenarlas, evitando registros duplicados, incompletos o contradictorios.
Deberán respetarse las reglas de negocio establecidas para el sistema. Por ejemplo:

- Un bus no podrá estar asignado a dos líneas simultáneamente.
- Un bus deberá mantener un parqueo asignado.
- Una estación deberá pertenecer a una municipalidad.
- Un acceso deberá pertenecer a una estación.
- La cantidad de buses asignados a una línea deberá respetar los límites establecidos.
- Cada acceso deberá contar con el control de seguridad correspondiente.

Las validaciones deberán realizarse tanto desde la aplicación como mediante las restricciones correspondientes de la base de datos cuando sea necesario.

El objetivo es garantizar que la información utilizada para la toma de decisiones sea confiable y coherente.
el sistema deberá rechazar cualquier operación que provoque una inconsistencia o viole una regla de negocio.
Criterio medible: 28 llaves foráneas, 39 restricciones CHECK, 5 triggers y 1 procedimiento almacenado; cada regla del enunciado tiene al menos una prueba automatizada que intenta violarla y comprueba el rechazo.

RNF-03. Disponibilidad
La plataforma deberá estar disponible durante el horario operativo del sistema de transporte, permitiendo que los operadores y administradores puedan consultar y registrar información cuando sea necesario.
Debido a que las estaciones dependerán de la plataforma para realizar diferentes actividades de control, deberán implementarse mecanismos destinados a reducir las interrupciones del servicio.
La solución deberá considerar procedimientos de recuperación ante fallos y mecanismos de respaldo que permitan restablecer la operación en caso de presentarse una interrupción.
La disponibilidad deberá ser especialmente importante para las funcionalidades relacionadas con el monitoreo de pasajeros y la generación de alertas, debido a que una interrupción podría retrasar la respuesta ante situaciones de saturación.
El objetivo es garantizar la continuidad del sistema durante la operación normal del Transmetro.
la plataforma deberá mantenerse operativa durante el horario definido para el servicio, registrando y documentando cualquier interrupción.
Criterio medible: disponibilidad mayor o igual al 99 % durante el horario de servicio (de 04:30 a 21:00, según los horarios oficiales cargados), medida con un monitor externo. En producción se usan planes de pago, porque en el plan gratuito Render suspende la aplicación tras 15 minutos sin uso y Supabase pausa el proyecto tras 7 días de inactividad.

RNF-04. Rendimiento
Las consultas y operaciones realizadas dentro de la plataforma deberán responder en tiempos adecuados para no afectar las actividades de los operadores de las estaciones.
Las consultas frecuentes, como búsqueda de estaciones, líneas, buses, parqueos y alertas, deberán ejecutarse de manera eficiente.
Las operaciones relacionadas con situaciones críticas deberán recibir especial atención. Por ejemplo, cuando una estación alcance el nivel de pasajeros establecido para generar una alerta, el sistema deberá procesar la información oportunamente para que el operador pueda tomar las acciones correspondientes.
El rendimiento deberá evaluarse mediante pruebas que permitan determinar el comportamiento de la aplicación bajo diferentes cantidades de usuarios y registros.
Poder evitar que los tiempos de espera del sistema afecten las operaciones del personal.
las operaciones de uso frecuente deberán ejecutarse dentro de los tiempos establecidos durante las pruebas de rendimiento.

Criterio medible: el 95 % de las operaciones frecuentes (consultas, registro de pase, actualización de aforo) responde en menos de 2 segundos con 50 usuarios simultáneos; la alerta de refuerzo se crea en la misma transacción que actualiza el aforo.

RNF-05. Escalabilidad

La solución deberá diseñarse de manera que pueda crecer conforme aumenten las necesidades del sistema de transporte.

La plataforma deberá permitir incorporar nuevas:

- Municipalidades.
- Líneas.
- Estaciones.
- Buses.
- Parqueos.
- Usuarios.
- Operadores.

El crecimiento de la información no deberá requerir una reconstrucción completa de la aplicación.

La arquitectura deberá permitir que posteriormente puedan incorporarse nuevas funcionalidades relacionadas con la operación del transporte sin afectar significativamente los módulos existentes.

Por ejemplo, si posteriormente se incorpora una nueva municipalidad al sistema, deberá ser posible registrarla y administrar sus líneas y estaciones utilizando la infraestructura existente.
proteger la inversión realizada y permitir que la solución acompañe el crecimiento futuro del Transmetro.

el sistema deberá permitir agregar nuevos recursos sin modificar de manera significativa la estructura principal de la aplicación.

Criterio medible: agregar una línea, estación o municipalidad no requiere cambios de código; la red actual de 8 líneas, 121 estaciones y 173 buses se cargó solo con datos.

RNF-06. Usabilidad

La interfaz deberá ser clara, intuitiva y fácil de utilizar por operadores y personal administrativo.

Las funciones deberán organizarse de manera lógica, utilizando menús, botones, formularios y mensajes claramente identificados.

Las operaciones frecuentes deberán requerir la menor cantidad de pasos posibles para evitar errores durante las actividades diarias.

Cuando el usuario realice una operación incorrecta, el sistema deberá mostrar un mensaje comprensible que indique el problema y, cuando sea posible, la forma de solucionarlo.

Por ejemplo, si un operador intenta asignar un bus que ya pertenece a otra línea, el sistema deberá mostrar un mensaje explicando que la operación no puede realizarse debido a que el bus ya se encuentra asignado.

El objetivo es facilitar la adopción del sistema y reducir errores provocados por una interfaz complicada.

un operador previamente capacitado deberá poder realizar las operaciones correspondientes a su puesto sin requerir conocimientos técnicos avanzados.
Criterio medible: un operador capacitado registra un pase o actualiza el aforo en un máximo de 3 clics desde su panel; todos los errores de base de datos se muestran con un mensaje en español.

RNF-07. Auditoría
El sistema deberá registrar las operaciones importantes realizadas por los usuarios para permitir conocer quién realizó una acción, cuándo la realizó y qué información modificó.
El registro de auditoría deberá contemplar, como mínimo:

- Usuario que realizó la operación.
- Fecha.
- Hora.
- Tipo de operación.
- Registro afectado.
- Acción realizada.

Deberán considerarse especialmente las operaciones relacionadas con:

- Asignación de buses.
- Cambio de parqueos.
- Modificación de líneas.
- Modificación de estaciones.
- Administración de usuarios.
- Modificación de información de pilotos.
- Gestión de personal de seguridad.
- Operaciones relacionadas con tarjetas de acceso.

La información de auditoría deberá estar disponible únicamente para usuarios que tengan autorización para consultarla.
El objetivo es proporcionar trazabilidad sobre las operaciones realizadas dentro de la plataforma. ante una modificación importante, el sistema deberá permitir identificar al usuario responsable y la fecha y hora en que se realizó.
Criterio medible: el 100 % de las operaciones del operador y de los cambios de trazo queda en bitacora_auditoria con usuario, fecha, datos anteriores y posteriores; meta pendiente: extenderlo a todos los módulos administrativos.

RNF-08. Respaldo
La información almacenada deberá contar con mecanismos de respaldo que permitan reducir el riesgo de pérdida de información.
Los respaldos deberán incluir información crítica relacionada con:

- Líneas.
- Estaciones.
- Buses.
- Parqueos.
- Pilotos.

- Usuarios.
- Personal de seguridad.
- Tarjetas, recargas y bitácora de auditoría.
- Alertas.
- Transacciones.

Se deberá establecer una política de respaldo que defina la frecuencia con la que se realizarán las copias y el procedimiento para recuperar la información.
Los respaldos deberán mantenerse en una ubicación diferente a la utilizada para la operación principal cuando la infraestructura disponible lo permita.
Además, deberán realizarse pruebas periódicas de restauración para comprobar que los respaldos pueden utilizarse correctamente.
minimizar el impacto de fallos técnicos, errores humanos o incidentes que puedan provocar pérdida de información. deberá existir un respaldo funcional y un procedimiento documentado para recuperar la información.
Criterio medible: respaldo diario automático (Supabase Pro, retención de 7 días) más un respaldo semanal con pg_dump guardado fuera de Supabase; una prueba de restauración al mes.

RNF-09. Mantenibilidad
La aplicación deberá desarrollarse bajo una arquitectura organizada y modular que facilite la corrección de errores, actualización y evolución del sistema.
El código deberá mantener una estructura clara y separada por responsabilidades, evitando que una modificación en un módulo afecte innecesariamente el funcionamiento de otros componentes.
La solución deberá contar con documentación suficiente para que un equipo técnico pueda comprender su funcionamiento y realizar mantenimiento posteriormente.
Las modificaciones deberán realizarse mediante procedimientos controlados que permitan identificar los cambios realizados y verificar que las funcionalidades existentes continúen funcionando correctamente.
reducir el tiempo y costo necesario para corregir errores y realizar actualizaciones. deberá ser posible modificar o actualizar un módulo sin provocar fallos en las funcionalidades independientes del sistema.
Criterio medible: estilo verificado con Laravel Pint y 146 pruebas ejecutadas en GitHub Actions en cada cambio; complejidad ciclomática promedio menor a 10 por método (actual: 1.96).

RNF-10. Compatibilidad
La aplicación web deberá ser accesible desde navegadores modernos utilizando las computadoras instaladas en las estaciones.
La interfaz deberá mantener un funcionamiento consistente independientemente del navegador compatible utilizado por los operadores.
Se deberán realizar pruebas de compatibilidad para verificar las principales funcionalidades del sistema, incluyendo:

- Inicio de sesión.
- Consulta de información.
- Registro de datos.
- Modificación de registros.
- Generación de reportes.
- Visualización de alertas.

La solución deberá evitar depender de configuraciones particulares del equipo que puedan impedir su utilización en otras estaciones.
garantizar que los operadores puedan utilizar la plataforma desde las computadoras destinadas para cada estación. las funcionalidades principales deberán operar correctamente en los navegadores definidos como compatibles para el proyecto.
Criterio medible: funcionamiento verificado en las dos últimas versiones de Google Chrome, Microsoft Edge y Mozilla Firefox, con resolución mínima de 1366 × 768.

RNF-11. Conectividad

La solución deberá contar con una infraestructura de comunicación que permita conectar las computadoras de las diferentes estaciones con los servicios centrales.
La arquitectura de conectividad deberá permitir que la información registrada en una estación pueda ser procesada y almacenada en los servicios centrales del sistema.
La comunicación deberá diseñarse considerando aspectos como:

- Disponibilidad de conexión.
- Seguridad de la comunicación.
- Capacidad de transmisión.
- Estabilidad.
- Recuperación ante interrupciones.

En caso de una pérdida temporal de conectividad, el sistema deberá informar al operador sobre el estado de la conexión y deberán definirse mecanismos para evitar, en la medida de lo posible, la pérdida de información.
La solución deberá contemplar una arquitectura que permita conectar las diferentes estaciones sin importar que se encuentren ubicadas en distintas zonas o municipalidades.
Este requerimiento adquiere especial importancia debido a que el planteamiento del proyecto establece que todavía no se ha determinado cuál será la mejor alternativa tecnológica para conectar las computadoras de las diferentes estaciones.
garantizar una comunicación confiable entre las estaciones y los servicios centrales de la plataforma. Las estaciones deberán poder comunicarse con los servicios centrales y transmitir información de manera segura y confiable durante la operación normal.
Decisión (punto 17): cada PC de estación se conecta por Internet mediante HTTPS (cifrado TLS) a la aplicación en Render, y solo la aplicación se conecta a la base de datos. No se requiere una red privada (WAN o VPN) entre estaciones. Si se pierde la conexión, la operación no se registra a medias: el operador recibe un mensaje y la repite al volver la conexión. Un modo sin conexión con almacenamiento local queda como mejora futura.

6. Estudios de Factibilidad

El estudio de factibilidad evalúa si la propuesta puede desarrollarse, operarse y mantenerse con los recursos disponibles y dentro del marco legal guatemalteco. Se analizan cinco dimensiones: técnica, operativa, económica, legal y ambiental. Los montos se expresan en quetzales; los servicios cobrados en dólares se convierten al tipo de cambio de referencia del Banco de Guatemala del 28 de septiembre de 2026 (Q7.63 por US$1).

6.1. Factibilidad técnica

La factibilidad técnica determina si existen la tecnología, la infraestructura y el conocimiento necesarios para construir y poner en marcha el sistema.

Recurso	Requerido	Disponibilidad	Evaluación
Lenguaje y framework	PHP 8.4 y Laravel 13	Software libre (licencia MIT); el equipo ya lo domina	Factible
Base de datos	PostgreSQL con triggers, procedimientos y RLS	PostgreSQL 16 en desarrollo y Supabase en la nube	Factible
Alojamiento	Servidor web con HTTPS	Render (Web Service con Docker)	Factible
Mapas	Mapa base y trazo de rutas	OpenStreetMap y Leaflet (abiertos); KML oficiales de las líneas	Factible
Control de versiones y pruebas	Repositorio e integración continua	GitHub y GitHub Actions con 146 pruebas automatizadas	Factible
PC de trabajo por estación (punto 16)	Navegador moderno, 4 GB de RAM, Internet	Equipo de oficina estándar disponible en el mercado local	Factible (requiere compra)
Conectividad (punto 17)	Enlace a Internet en cada estación	Proveedores locales de Internet fijo y 4G	Factible
Molinetes	Lector de tarjeta conectado al sistema	En esta versión el pase se registra desde el panel del operador; la conexión directa con el molinete requiere la API del fabricante	Parcial (fase 2)

Respuesta al punto 17 (cómo conectar las máquinas): se recomienda una arquitectura centralizada en la nube. Cada PC de estación se conecta por Internet mediante HTTPS (cifrado TLS) a la aplicación en Render, y solo la aplicación se conecta a la base de datos en Supabase. Así no es necesario montar una red privada (WAN o VPN) entre estaciones, se reduce el costo de infraestructura y una estación nueva funciona con un navegador y conexión a Internet. Como respaldo, cada estación puede contar con un módem 4G.

Prototipo funcional: el sistema ya cuenta con las 8 líneas oficiales (1, 2, 6, 7, 12, 13, 18 y Ruta 5), 121 estaciones, 173 buses, 5 reglas de negocio en triggers, un procedimiento almacenado y 146 pruebas automatizadas aprobadas, lo que demuestra que la solución es técnicamente viable.

Conclusión: el proyecto es técnicamente factible.

6.2. Factibilidad operativa

La factibilidad operativa evalúa si el personal del Transmetro podrá usar el sistema y si este se integra a su forma de trabajo.

- Usuarios: administradores municipales y operadores de estación. Los operadores ya contarán con una PC según el punto 16; el sistema solo requiere conocimientos básicos de navegador.
- Interfaz en español, con menús por rol, mensajes de error comprensibles (por ejemplo, al intentar exceder la flota de una línea) y un panel de estación en una sola pantalla.
- Las reglas de negocio se aplican automáticamente en la base de datos, lo que reduce los controles manuales.
- Capacitación de 24 horas para administradores y operadores, incluida en la propuesta monetaria, con manual de usuario.
- Resistencia al cambio: se mitiga con un piloto en las Líneas 1 y 2 (14 estaciones) antes de extender el sistema a toda la red.
- Soporte: el código está documentado, versionado en GitHub y cubierto por pruebas, lo que facilita el mantenimiento.

Riesgo operativo	Probabilidad	Impacto	Mitigación
Caída de Internet en una estación	Media	Alto	Enlace 4G de respaldo; la operación se repite al volver la conexión
Datos ingresados con errores	Media	Medio	Validaciones en formularios y restricciones en la base de datos
Rotación de operadores	Media	Bajo	Manual de usuario y capacitación corta
Suspensión por inactividad en planes gratuitos	Alta	Alto	Usar en producción Render Starter y Supabase Pro

Conclusión: el proyecto es operativamente factible.

6.3. Factibilidad económica

La factibilidad económica compara la inversión y los costos de operación con los beneficios esperados. El detalle de los costos está en la sección 7 (Propuesta Monetaria).

6.3.1. Costos

Concepto	Monto	Tipo
Desarrollo del software (puntos de función)	Q99,010.63	Inversión única
Integración: análisis, pruebas QA y capacitación	Q13,600.00	Inversión única
IVA (12 %) sobre la inversión	Q13,513.28	Inversión única
Inversión total del software	**Q126,123.90**	
Nube: Render Starter + Supabase Pro + dominio	Q3,044.37	Anual
Mantenimiento correctivo y evolutivo (10 % del desarrollo)	Q9,901.06	Anual
Costo anual de operación	**Q12,945.43**	

Las licencias de software tienen costo cero porque todo el stack es de código abierto. El equipo de las estaciones (PC, UPS e Internet) es una inversión de infraestructura que la municipalidad necesita de todas formas por el punto 16, por lo que se presenta aparte en la sección 7.8.

6.3.2. Beneficios estimados

Los beneficios se calcularon con supuestos conservadores que deben validarse con la municipalidad:

Beneficio	Supuesto	Monto anual
Menos horas de consolidación manual de información (estaciones, buses, personal y reportes)	3 personas × 40 h/mes × Q40/h × 12 meses	Q57,600.00
Menos papel, impresión y formularios de control	Q500 al mes	Q6,000.00
Total de beneficios cuantificables		**Q63,600.00**

Además hay beneficios no cuantificados: alertas de refuerzo en el momento en que se superan los límites, control de la flota mínima y máxima por línea, trazabilidad de pagos y cambios, y reportes inmediatos para la toma de decisiones.

6.3.3. Indicadores financieros (horizonte de 5 años, tasa de descuento del 10 %)

Indicador	Valor	Interpretación
Flujo neto anual	Q50,654.57	Beneficios menos costo anual de operación
Valor actual neto (VAN)	Q65,896.76	Positivo: la inversión se recupera con ganancia
Relación beneficio/costo	1.38	Mayor que 1: cada quetzal invertido devuelve más de un quetzal
Período de recuperación	2.5 años	Dentro del horizonte de evaluación

Conclusión: con los supuestos indicados, el proyecto es económicamente factible. Si los beneficios reales fueran menores, el punto de equilibrio se alcanza con un beneficio anual de Q46,216.60.

6.4. Factibilidad legal

Se revisó el marco legal guatemalteco aplicable al manejo de la información, a la contratación del servicio y al uso de software y datos de terceros.

Norma	Aplicación al proyecto
Constitución Política de la República de Guatemala, artículos 24 y 31	Protege los documentos y registros de las personas y su derecho a conocer la información que de ellas consta en archivos del Estado. Los datos del personal y de los titulares de tarjetas se guardan con acceso restringido.
Ley de Acceso a la Información Pública, Decreto 57-2008	La municipalidad es sujeto obligado. Los datos personales sensibles (DPI, dirección, antecedentes) son información confidencial y solo los consulta el personal autorizado; los reportes públicos no incluyen datos personales.
Ley para el Reconocimiento de las Comunicaciones y Firmas Electrónicas, Decreto 47-2008	Da validez a los registros electrónicos, como transacciones de pago y bitácora de auditoría.
Código Municipal, Decreto 12-2002	El transporte público urbano es competencia de la municipalidad, que es la entidad contratante y dueña de la información.
Ley de Tránsito, Decreto 132-96	Exige licencia de conducir vigente a los pilotos; el sistema registra número y tipo de licencia.
Ley de Derecho de Autor y Derechos Conexos, Decreto 33-98	Protege el software como obra. El contrato debe definir a quién pertenece el código fuente.
Ley de Contrataciones del Estado, Decreto 57-92	Por el monto de la propuesta (Q129,534), la contratación corresponde a la modalidad de cotización (más de Q90,000 y hasta Q900,000) a través de Guatecompras.
Licencias del software utilizado	Laravel, Tailwind y Pest (MIT), PHP (PHP License), PostgreSQL (PostgreSQL License), Leaflet (BSD-2). Todas permiten uso sin costo. Los datos de OpenStreetMap (ODbL) exigen mostrar el crédito “© OpenStreetMap”, que ya aparece en el mapa.
Servicios en la nube	Los datos se alojan en servidores de Supabase fuera de Guatemala; el contrato con la municipalidad debe autorizarlo y exigir cifrado y respaldos.

Conclusión: el proyecto es legalmente factible, siempre que el contrato defina la propiedad del código, se publique un aviso de privacidad para los titulares de tarjetas y se mantenga el crédito de OpenStreetMap.

6.5. Factibilidad ambiental

Se evaluó el impacto ambiental del desarrollo y de la operación del sistema, en el marco de la Ley de Protección y Mejoramiento del Medio Ambiente (Decreto 68-86).

Aspecto	Impacto	Medida
Uso de papel	Positivo	Reportes, formularios y bitácoras digitales; los reportes se imprimen solo cuando es necesario.
Manejo de efectivo	Positivo	Tarjeta Ciudadana y pago sin contacto: menos traslado de valores.
Emisiones de los buses	Positivo	Refuerzos solo cuando la estación supera el 150 % y espera extra cuando el bus va casi vacío: viajes más llenos y menos emisiones por pasajero.
Energía de servidores	Bajo	Nube compartida en centros de datos eficientes; no se instala un servidor propio con aire acondicionado.
Energía de las PC de estación	Moderado	Equipos con certificación de eficiencia energética y suspensión fuera del horario de servicio.
Residuos electrónicos	Moderado	Al final de su vida útil, PC, UPS y baterías se entregan a gestores autorizados por el MARN.

Conclusión: el impacto ambiental es bajo y en su mayoría positivo; el proyecto es ambientalmente factible.

6.6. Resumen del estudio de factibilidad

Dimensión	Resultado	Condición principal
Técnica	Factible	Prototipo funcional con pruebas; integración con molinetes en fase 2
Operativa	Factible	Capacitación y piloto en las Líneas 1 y 2
Económica	Factible	VAN de Q65,897 con los supuestos de beneficio indicados
Legal	Factible	Contrato con propiedad del código y aviso de privacidad
Ambiental	Factible	Manejo responsable de los residuos electrónicos

7. Propuesta Monetaria

La propuesta monetaria se calculó con el método de Puntos de Función (FPA-IFPUG), igual que la estimación original, pero ajustada al alcance real del enunciado y del sistema construido. Primero se presenta la revisión de la estimación original y después la estimación corregida.

7.1. Revisión de la estimación original

Las fórmulas de la estimación original están bien aplicadas (PFSA 288 → PFA 362.88 → 2,177.28 H/H → 6.48 meses → Q79,010.00; total Q117,120.00). Las correcciones se deben al contenido:

No.	Hallazgo en la estimación original	Corrección
1	Ítem 14: inyección de bus al llegar al 50 % del aforo	La alerta se genera al 150 % del aforo (50 % más de su capacidad, punto 14)
2	Ítem 15: buses en circulación ≤ paradas de la ruta	La regla es mínimo n y máximo 2n buses por línea (punto 5)
3	Ítems 17 a 21: bahías, notificación push y estacionamiento dinámico por orden de llegada	Contradicen los puntos 6 y 7: cada bus tiene un parqueo fijo que puede cambiar, pero nunca quedarse sin él
4	Ítem 31: cobro en efectivo con monedas de Q1.00	Se elimina: el molinete solo acepta Tarjeta Ciudadana o tarjeta bancaria sin contacto
5	Ítems 34 y 35: autenticación en Vercel y modo sin conexión	La arquitectura es Laravel en Render con PostgreSQL en Supabase; el modo sin conexión queda como mejora futura
6	Ítems 5 y 12: pantalla del piloto y GPS	Se pasan a la fase 2 (no forman parte del enunciado)
7	Faltan funciones construidas	Se agregan Tarjeta Ciudadana y recargas, reportes, mapa e importación KML, horarios y bitácora
8	ILF = 12; EIF = GPS, pasarela y torniquetes	ILF = 14 grupos lógicos de datos; EIF = mapas KML, OpenStreetMap, OSRM y molinete o pasarela
9	“Otros costos” (Q1,250) se sumaron una sola vez	Son gastos mensuales (transporte, alimentación y conectividad); se multiplican por los meses del proyecto
10	“Horas promedio por PF: 6 por día”	Son 6 horas por punto de función, no por día
11	Hosting en Vercel (Q2,160/año) y Render (Q4,200/año)	Se elimina Vercel; Render Starter US$7/mes y Supabase Pro US$25/mes
12	Hardware solo para 4 accesos, 4 estaciones y 4 buses	El punto 16 exige una PC por estación; se propone un piloto de 14 estaciones (Líneas 1 y 2) como inversión municipal
13	Suma de factores de ajuste = 61	Baja a 57: rendimiento, configuración, tasa de transacción y procesamiento complejo bajan un grado porque no hay GPS ni control de bahías en tiempo real

7.2. Clasificación corregida de las funciones

Estado: I = implementada en el prototipo; P = pendiente para el siguiente sprint.

No.	Módulo	Función	Tipo	Estado
1	1. Infraestructura	Registrar y actualizar municipalidades	EI	I

No.	Módulo	Función	Tipo	Estado
2	1. Infraestructura	Registrar y actualizar líneas (código, nombre, tipo de servicio, tarifa, color)	EI	I
3	1. Infraestructura	Asignar estaciones a una línea en orden (distancia calculada por trigger)	EI	I
4	1. Infraestructura	Registrar y actualizar estaciones (dirección, aforo, coordenadas)	EI	I
5	1. Infraestructura	Registrar accesos (mínimo 2) con su molinete	EI	I
6	1. Infraestructura	Registrar parqueos por estación	EI	I
7	1. Infraestructura	Registrar horarios de servicio por línea	EI	I
8	1. Infraestructura	Editar el trazo de la línea e importar mapa KML/KMZ oficial	EI	I
9	2. Flota	Registrar buses y asignarlos a una línea (mínimo n, máximo 2n)	EI	I
10	2. Flota	Cambiar el parqueo de un bus sin dejarlo sin parqueo (sp_cambiar_parqueo_bus)	EI	I
11	2. Flota	Registrar viajes y visitas a estaciones (llegada, salida, ocupación)	EI	P
12	3. Personal	Registrar pilotos (historial educativo, residencia, comunicación)	EI	I
13	3. Personal	Registrar seguridad y asignarlo a un acceso (mínimo 1 por acceso)	EI	I
14	3. Personal	Registrar operadores y asignarlos a la PC de su estación	EI	I
15	4. Tarjeta y pagos	Emitir Tarjeta Ciudadana (Q20.00 con 5 viajes gratis)	EI	I
16	4. Tarjeta y pagos	Recargar tarjeta (Q5.00 a Q200.00)	EI	I
17	4. Tarjeta y pagos	Bloquear o reactivar una tarjeta	EI	I
18	4. Tarjeta y pagos	Registrar pase en molinete (Tarjeta Ciudadana o tarjeta bancaria)	EI	I
19	5. Operación de estación	Actualizar el aforo actual de la estación	EI	I
20	5. Operación de estación	Atender alerta de refuerzo (solicitar bus)	EI	I
21	6. Seguridad	Iniciar y cerrar sesión con control de rol	EI	I
22	5. Operación de estación	Alerta automática de refuerzo al superar 150 % del aforo	EO	I
23	2. Flota	Cálculo de espera extra de 5 min cuando la ocupación es menor a 25 %	EO	I
24	7. Reportes	Reporte de estaciones (líneas, accesos, parqueos, aforo)	EO	I
25	7. Reportes	Reporte de líneas (estaciones, distancia, flota mínima y máxima)	EO	I
26	7. Reportes	Reporte de buses asignados por línea	EO	I
27	7. Reportes	Reporte de recorridos (viajes)	EO	I
28	7. Reportes	Reporte de visitas y ocupación por estación	EO	I
29	7. Reportes	Exportación CSV e impresión de reportes	EO	I
30	7. Reportes	Reporte de alertas generadas y atendidas	EO	P
31	7. Reportes	Reporte de recaudación por método de pago	EO	P
32	7. Reportes	Reporte de personal asignado	EO	P
33	1. Infraestructura	Mapa de líneas y estaciones (OpenStreetMap)	EQ	I
34	1. Infraestructura	Consultar línea: estaciones en orden, accesos y horarios	EQ	I
35	1. Infraestructura	Consultar estación: accesos, molinetes, guardias y parqueos	EQ	I
36	2. Flota	Consultar buses y parqueos	EQ	I
37	3. Personal	Consultar expedientes del personal	EQ	I
38	4. Tarjeta y pagos	Consultar tarjeta (saldo, viajes gratis, recargas)	EQ	I
39	5. Operación de estación	Panel del operador (aforo, alertas y buses de su estación)	EQ	I

No.	Módulo	Función	Tipo	Estado
40	6. Seguridad	Panel del administrador con indicadores	EQ	I
41	6. Seguridad	Consultar bitácora de auditoría	EQ	P

Resumen	Cantidad
Entradas externas (EI)	21
Salidas externas (EO)	11
Consultas externas (EQ)	9
Total de transacciones	**41**
Implementadas / pendientes	36 / 5

7.3. Archivos lógicos

Archivos lógicos internos (ILF)	Archivos de interfaz externa (EIF)
1. Municipio	1. Mapas KML/KMZ oficiales de las líneas
2. Línea (con horarios y orden de estaciones)	2. Teselas de OpenStreetMap
3. Estación	3. Servicio de ajuste a calles OSRM
4. Acceso y molinete	4. Lector de molinete / pasarela bancaria (integración futura)
5. Parqueo	
6. Bus	
7. Empleado (piloto, seguridad, operador)	
8. Usuario y rol	
9. Tarjeta	
10. Recarga	
11. Transacción de pago	
12. Alerta del sistema	
13. Viaje y visitas	
14. Bitácora de auditoría	

7.4. Cálculo de puntos de función

Componente	Cantidad	Peso (media)	Subtotal
Entradas (EI)	21	4	84
Salidas (EO)	11	5	55
Consultas (EQ)	9	4	36
Archivos internos (ILF)	14	10	140
Archivos externos (EIF)	4	7	28
Puntos de función sin ajustar (PFSA)			**343**

No.	Factor de ajuste	Grado (0-5)
1	Comunicación de datos	5
2	Proceso distribuido	4
3	Objetivos de rendimiento	4
4	Configuración del equipamiento	3
5	Tasa de transacción	4
6	Entrada de datos en línea	5
7	Interfaz con el usuario	4
8	Actualizaciones en línea	5
9	Procesamiento complejo	4
10	Reusabilidad del código	4
11	Conversión e instalación	3
12	Facilidad de operación	4
13	Instalación múltiple	4
14	Facilidad de cambios	4
	Suma de factores	**57**

Resultado	Valor	Fórmula
Factor de ajuste (FA)	1.22	0.65 + 0.01 × suma de factores
Puntos de función ajustados (PFA)	418.46	PFSA × FA
Horas hombre (H/H)	2,510.76	PFA × 6 horas por PF
Horas por trabajador	1,255.38	H/H ÷ 2 trabajadores
Tiempo (meses)	7.47	Horas ÷ (8 h × 21 días)
Líneas de código estimadas	20,923	PFA × 50 LOC (lenguaje de 4.ª generación)
Sueldos	Q89,670.00	Meses × Q6,000 × 2 trabajadores
Otros costos	Q9,340.63	Meses × Q1,250 (transporte, alimentación y conectividad)
Costo de desarrollo	**Q99,010.63**	Sueldos + otros costos
Precio por hora	Q39.43	Costo ÷ H/H

7.5. Costos del proceso de integración

Actividad	Horas	Costo por hora	Total
Análisis de la operación y arquitectura (Render + Supabase)	55	Q100.00	Q5,500.00
Pruebas de software QA (funcionales, carga, seguridad y SonarQube Cloud)	60	Q85.00	Q5,100.00
Capacitación de administradores y operadores	24	Q125.00	Q3,000.00
Total de integración			**Q13,600.00**

7.6. Costos de operación en la nube (anual)

Servicio	US$ por año	Q por año
Render – Web Service Starter (512 MB, siempre activo)	84.00	Q640.92
Supabase – plan Pro (respaldos diarios, sin pausa)	300.00	Q2,289.00
Dominio .com y certificado TLS (incluido en Render)	15.00	Q114.45
Total anual	**399.00**	**Q3,044.37**

Tipo de cambio: Q7.63 por US$1 (Banguat, 28/09/2026). Licencias de IDE (VS Code), gestor de base de datos (PostgreSQL), framework y análisis de calidad (SonarQube Cloud, plan gratuito para repositorios privados de hasta 50,000 líneas de código): Q0.00. Los planes gratuitos de Render y Supabase sirven para desarrollo, pero no para producción, porque suspenden el servicio por inactividad y no incluyen respaldos.

7.7. Resumen de la propuesta

Concepto	Monto
Desarrollo del software	Q99,010.63
Proceso de integración (análisis, pruebas y capacitación)	Q13,600.00
Operación en la nube, primer año	Q3,044.37
Subtotal	**Q115,655.00**
IVA (12 %)	Q13,878.60
Total de la propuesta	**Q129,533.59**
Costo recurrente desde el segundo año (nube + mantenimiento, sin IVA)	Q12,945.43

El tiempo estimado de desarrollo es de 7.5 meses con 2 desarrolladores. Como referencia, las funciones ya implementadas en el prototipo suman 320 PFSA (390.40 PFA), el 93.3 % del total estimado.

7.8. Inversión en equipo de estaciones (a cargo de la municipalidad)

El punto 16 requiere una PC de trabajo en cada estación. Este equipo no forma parte del precio del software; se presenta como referencia para la municipalidad. Los precios son referenciales y deben cotizarse.

Elemento	Precio unitario	Piloto Líneas 1 y 2 (14 estaciones)	Red completa (121 estaciones)
PC de trabajo	Q4,500.00	Q63,000.00	Q544,500.00
UPS	Q750.00	Q10,500.00	Q90,750.00
Inversión en equipo		**Q73,500.00**	**Q635,250.00**
Internet por estación (anual)	Q3,600.00	Q50,400.00	Q435,600.00

Fase 2 opcional, por cotizar: tablets para la consola del piloto (referencia Q1,200 por bus) e integración directa con los lectores de los molinetes, que depende de la API del fabricante.

7.9. Forma de pago

Hito	Porcentaje	Monto
Firma del contrato	30 %	Q38,860.08
Entrega del piloto en las Líneas 1 y 2	40 %	Q51,813.44
Carta de aceptación del proyecto	30 %	Q38,860.08

8. Modelo Entidad Relación

El modelo entidad relación se generó directamente desde la base de datos final, después de aplicar todas las migraciones del proyecto, por lo que coincide con el script de la sección 9 y con la aplicación en funcionamiento. Reemplaza la versión anterior elaborada en DBeaver, a la que le faltaban una tabla y 14 columnas agregadas durante la programación.

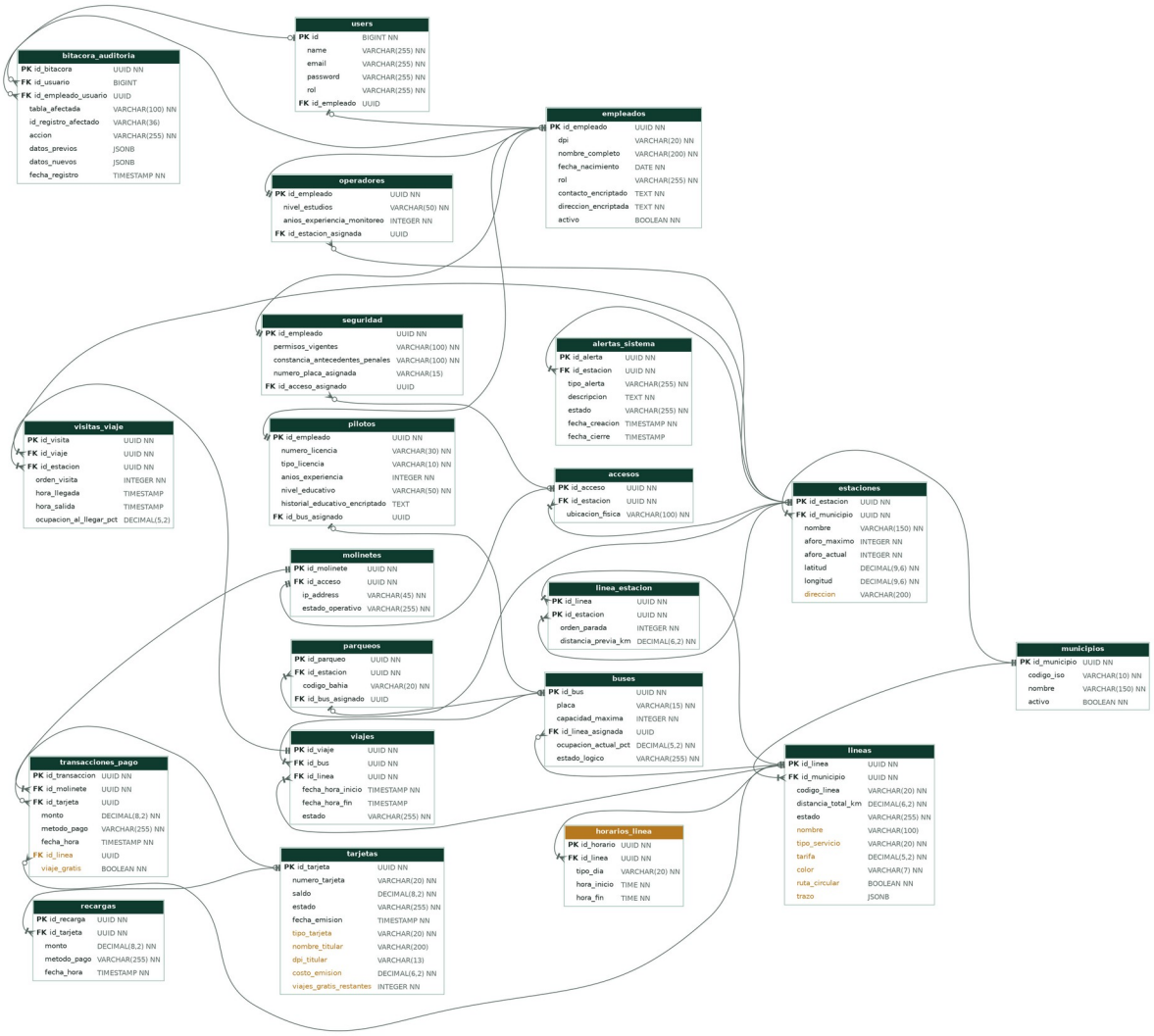


Figura 1. Modelo entidad relación del Sistema de Control Transmetro (PostgreSQL).

Convenciones: PK = llave primaria, FK = llave foránea, NN = no nulo. La tabla con encabezado dorado y los atributos en dorado son los que se agregaron respecto de la versión anterior del modelo. Las relaciones usan notación de pata de gallo: la línea simple indica “uno” y la pata de gallo indica “muchos”; el círculo indica que la relación es opcional.

8.1. Cambios respecto de la versión anterior

Tabla	Cambio	Motivo
horarios_linea	Tabla nueva	Horario oficial por tipo de día (RF-22)
lineas	nombre, tipo_servicio, tarifa, color, ruta_circular, trazo	Datos oficiales de las líneas, tarifa del pase y trazo del mapa (RF-02, RF-20)
estaciones	direccion; coordenadas en latitud y longitud	Ubicación en el mapa (RF-03)
tarjetas	tipo_tarjeta, nombre_titular, dpi_titular, costo_emision, viajes_gratis_restantes	Tarjeta Ciudadana: Q20 con 5 viajes gratis (RF-21)
transacciones_pago	id_linea, viaje_gratis	Tarifa por línea y descuento de viajes gratis (RF-14)

8.2. Relaciones y reglas del enunciado

Relación	Cardinalidad	Regla (punto del enunciado)
municipios – lineas / estaciones	1 : N	Cada línea y estación pertenece a una municipalidad (11)
lineas – estaciones (linea_estacion)	N : M	Orden de las estaciones y distancia entre ellas (1, 2, 12, 13)
estaciones – accesos	1 : N (mínimo 2)	Varios accesos por estación; un acceso, una estación (3)
accesos – molinetes	1 : 1	Un molinete propio por acceso
accesos – seguridad	1 : N (mínimo 1)	Al menos un guardia por acceso (10)
lineas – buses	1 : N (de n a 2n)	Un bus en una sola línea o en ninguna (4, 5)
estaciones – parqueos – buses	1 : N y 1 : 1	Cada bus tiene un parqueo y puede cambiarlo (6, 7)
empleados – pilotos / seguridad / operadores	1 : 1	Especialización del personal (9, 10, 16)
buses – viajes – visitas_viaje	1 : N	Recorrido real y visitas a estaciones (1, 15)
estaciones – alertas_sistema	1 : N	Alerta de refuerzo al 150 % del aforo (14)
tarjetas – recargas / transacciones_pago	1 : N	Métodos de pago y tarjeta de acceso (18)
molinetes – transacciones_pago	1 : N	Pase registrado en el molinete (18)
users – bitacora_auditoria	1 : N	Trazabilidad de los cambios

9. Script de la estructura de base de datos

El script crea todas las estructuras de la base de datos en PostgreSQL 15 o superior. Se generó desde la base de datos resultante de las migraciones de Laravel y se validó ejecutándolo en una base vacía, sin errores. Equivale a ejecutar php artisan migrate.

Objeto	Cantidad	Detalle
Tablas del dominio	20	Más la tabla users de Laravel, a la que se agregan rol e id_empleado
Llaves foráneas	28	Con ON UPDATE CASCADE y la acción de borrado adecuada para cada caso
Restricciones CHECK	39	Valores permitidos, rangos (recargas de Q5 a Q200, ocupación de 0 a 100 %) y reglas de pago
Triggers	5	Mínimo 2 accesos, mínimo 1 guardia, máximo 2n buses, alerta al 150 %, distancia total de la línea
Procedimiento almacenado	1	sp_cambiar_parqueo_bus: cambio de parqueo sin dejar al bus sin parqueo
Row Level Security	21 tablas	Bloquea el acceso directo desde la API pública de Supabase

Forma de ejecución: en DBeaver, abrir el archivo Script_Creacion_Estructuras_BD_PostgreSQL.sql sobre la conexión de Supabase y usar “Ejecutar script” (Alt + X); o desde la terminal: psql "cadena de conexión" -f Script_Creacion_Estructuras_BD_PostgreSQL.sql. La tabla users debe existir antes (se crea con la migración inicial de Laravel).

9.1. Script de creación de estructuras (DDL)

```
-- ======================================================================
-- PROYECTO ACS - TRANSMETRO
-- Script de creacion de estructuras de base de datos (DDL)
-- Motor: PostgreSQL 15+ (Supabase)
-- Autor: Wendy Celeste Estrada
-- Fecha: 2026-09-30
-- Equivale a las migraciones de Laravel en database/migrations.
-- Requiere la tabla users de Laravel (se crea con php artisan migrate).
-- ======================================================================

-- ----------------------------------------------------------------------
-- 1. MUNICIPIOS
--    Punto 11: cada linea y estacion pertenece a una municipalidad.
-- ----------------------------------------------------------------------
CREATE TABLE municipios (
    id_municipio  UUID NOT NULL DEFAULT gen_random_uuid(),
    codigo_iso    VARCHAR(10) NOT NULL,
    nombre        VARCHAR(150) NOT NULL,
    activo        BOOLEAN NOT NULL DEFAULT true,
    CONSTRAINT municipios_pkey PRIMARY KEY (id_municipio),
    CONSTRAINT uq_municipios_codigo_iso UNIQUE (codigo_iso)
);

-- ----------------------------------------------------------------------
-- 2. LINEAS
--    Puntos 11 y 13. distancia_total_km se recalcula con trg_linea_estacion_distancia. Tarifa: Q1.00 paradas
continuas, Q2.00 directo. Color y ruta circular para el mapa; trazo (JSONB) = recorrido por las calles tomado del
mapa oficial.
-- ----------------------------------------------------------------------
CREATE TABLE lineas (
    id_linea            UUID NOT NULL DEFAULT gen_random_uuid(),
    id_municipio        UUID NOT NULL,
    codigo_linea        VARCHAR(20) NOT NULL,
    distancia_total_km  DECIMAL(6,2) NOT NULL DEFAULT '0'::numeric,
    estado              VARCHAR(255) NOT NULL DEFAULT 'activa',
    nombre              VARCHAR(100),
    tipo_servicio       VARCHAR(20) NOT NULL DEFAULT 'paradas_continuas',
    tarifa              DECIMAL(5,2) NOT NULL DEFAULT 1.00,
    color               VARCHAR(7) NOT NULL DEFAULT '#155240',
    ruta_circular       BOOLEAN NOT NULL DEFAULT false,
    trazo               JSONB,
    CONSTRAINT lineas_pkey PRIMARY KEY (id_linea),
    CONSTRAINT uq_lineas_codigo UNIQUE (codigo_linea),
```

```
    CONSTRAINT fk_lineas_municipio FOREIGN KEY (id_municipio) REFERENCES municipios(id_municipio) ON UPDATE CASCADE
ON DELETE RESTRICT,
    CONSTRAINT chk_lineas_color CHECK (color ~ '^#[0-9A-Fa-f]{6}$'),
    CONSTRAINT chk_lineas_distancia CHECK (distancia_total_km >= 0),
    CONSTRAINT chk_lineas_tarifa CHECK (tarifa >= 0),
    CONSTRAINT chk_lineas_tipo_servicio CHECK (tipo_servicio IN ('paradas_continuas', 'directo')),
    CONSTRAINT chk_lineas_trazo CHECK (trazo IS NULL OR jsonb_typeof(trazo) = 'array'),
    CONSTRAINT lineas_estado_check CHECK (estado IN ('activa', 'inactiva', 'mantenimiento'))
);

-- ---------------------------------------------------------------------
-- 3. ESTACIONES
--    Punto 11. Direccion y coordenadas latitud/longitud para el mapa.
-- ---------------------------------------------------------------------
CREATE TABLE estaciones (
    id_estacion   UUID NOT NULL DEFAULT gen_random_uuid(),
    id_municipio  UUID NOT NULL,
    nombre        VARCHAR(150) NOT NULL,
    aforo_maximo  INTEGER NOT NULL,
    aforo_actual  INTEGER NOT NULL DEFAULT 0,
    latitud       DECIMAL(9,6) NOT NULL,
    longitud      DECIMAL(9,6) NOT NULL,
    direccion     VARCHAR(200),
    CONSTRAINT estaciones_pkey PRIMARY KEY (id_estacion),
    CONSTRAINT fk_estaciones_municipio FOREIGN KEY (id_municipio) REFERENCES municipios(id_municipio) ON UPDATE
CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_estaciones_aforo CHECK (aforo_maximo > 0 AND aforo_actual >= 0),
    CONSTRAINT chk_estaciones_coordenadas CHECK (latitud BETWEEN -90 AND 90 AND longitud BETWEEN -180 AND 180)
);

-- ---------------------------------------------------------------------
-- 4. LINEA_ESTACION
--    Puntos 1, 2 y 12: estaciones en orden por linea y distancia desde la parada anterior.
-- ---------------------------------------------------------------------
CREATE TABLE linea_estacion (
    id_linea             UUID NOT NULL,
    id_estacion          UUID NOT NULL,
    orden_parada         INTEGER NOT NULL,
    distancia_previa_km  DECIMAL(6,2) NOT NULL DEFAULT '0'::numeric,
    CONSTRAINT linea_estacion_pkey PRIMARY KEY (id_linea, id_estacion),
    CONSTRAINT uq_linea_orden UNIQUE (id_linea, orden_parada),
    CONSTRAINT fk_linea_estacion_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE
CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_linea_estacion_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT chk_linea_estacion_distancia CHECK (distancia_previa_km >= 0),
    CONSTRAINT chk_linea_estacion_orden CHECK (orden_parada >= 1)
);
CREATE INDEX idx_linea_estacion_estacion ON linea_estacion (id_estacion);

-- ---------------------------------------------------------------------
-- 5. HORARIOS_LINEA
--    Horario de servicio por linea (lunes a viernes, sabado, domingo y feriados).
-- ---------------------------------------------------------------------
CREATE TABLE horarios_linea (
    id_horario   UUID NOT NULL DEFAULT gen_random_uuid(),
    id_linea     UUID NOT NULL,
    tipo_dia     VARCHAR(20) NOT NULL,
    hora_inicio  TIME WITHOUT TIME ZONE NOT NULL,
    hora_fin     TIME WITHOUT TIME ZONE NOT NULL,
    CONSTRAINT horarios_linea_pkey PRIMARY KEY (id_horario),
    CONSTRAINT uq_horarios_linea_dia UNIQUE (id_linea, tipo_dia),
    CONSTRAINT fk_horarios_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT chk_horarios_rango CHECK (hora_fin > hora_inicio),
    CONSTRAINT chk_horarios_tipo_dia CHECK (tipo_dia IN ('lunes_viernes', 'sabado', 'domingo_feriado'))
);

-- ---------------------------------------------------------------------
-- 6. BUSES
--    Puntos 4, 14 y 15: bus en una sola linea o ninguna, capacidad y ocupacion.
-- ---------------------------------------------------------------------
CREATE TABLE buses (
    id_bus                UUID NOT NULL DEFAULT gen_random_uuid(),
    placa                 VARCHAR(15) NOT NULL,
    capacidad_maxima      INTEGER NOT NULL,
    id_linea_asignada     UUID,
    ocupacion_actual_pct  DECIMAL(5,2) NOT NULL DEFAULT '0'::numeric,
    estado_logico         VARCHAR(255) NOT NULL DEFAULT 'fuera_de_servicio',
    CONSTRAINT buses_pkey PRIMARY KEY (id_bus),
    CONSTRAINT uq_buses_placa UNIQUE (placa),
    CONSTRAINT fk_buses_linea FOREIGN KEY (id_linea_asignada) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON
DELETE SET NULL,
    CONSTRAINT buses_estado_logico_check CHECK (estado_logico IN ('en_ruta', 'en_espera', 'fuera_de_servicio',
'en_mantenimiento')),
    CONSTRAINT chk_buses_capacidad CHECK (capacidad_maxima > 0),
    CONSTRAINT chk_buses_ocupacion CHECK (ocupacion_actual_pct BETWEEN 0 AND 100)
);
CREATE INDEX idx_buses_linea ON buses (id_linea_asignada);

-- ---------------------------------------------------------------------
-- 7. ACCESOS
--    Punto 3: varios accesos por estacion, un acceso pertenece a una sola estacion.
-- ---------------------------------------------------------------------
CREATE TABLE accesos (
    id_acceso         UUID NOT NULL DEFAULT gen_random_uuid(),
    id_estacion       UUID NOT NULL,
```

```
    ubicacion_fisica  VARCHAR(100) NOT NULL,
    CONSTRAINT accesos_pkey PRIMARY KEY (id_acceso),
    CONSTRAINT fk_accesos_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE RESTRICT
);
CREATE INDEX idx_accesos_estacion ON accesos (id_estacion);

-- ---------------------------------------------------------------------
-- 8. MOLINETES
--    Un molinete propio por acceso (1:1).
-- ---------------------------------------------------------------------
CREATE TABLE molinetes (
    id_molinete       UUID NOT NULL DEFAULT gen_random_uuid(),
    id_acceso         UUID NOT NULL,
    ip_address        VARCHAR(45) NOT NULL,
    estado_operativo  VARCHAR(255) NOT NULL DEFAULT 'activo',
    CONSTRAINT molinetes_pkey PRIMARY KEY (id_molinete),
    CONSTRAINT uq_molinetes_acceso UNIQUE (id_acceso),
    CONSTRAINT fk_molinetes_acceso FOREIGN KEY (id_acceso) REFERENCES accesos(id_acceso) ON UPDATE CASCADE ON
DELETE RESTRICT,
    CONSTRAINT molinetes_estado_operativo_check CHECK (estado_operativo IN ('activo', 'inactivo', 'mantenimiento'))
);

-- ---------------------------------------------------------------------
-- 9. PARQUEOS
--    Puntos 6 y 7: un bus nunca ocupa dos parqueos (UNIQUE) y cambia con sp_cambiar_parqueo_bus.
-- ---------------------------------------------------------------------
CREATE TABLE parqueos (
    id_parqueo       UUID NOT NULL DEFAULT gen_random_uuid(),
    id_estacion      UUID NOT NULL,
    codigo_bahia     VARCHAR(20) NOT NULL,
    id_bus_asignado  UUID,
    CONSTRAINT parqueos_pkey PRIMARY KEY (id_parqueo),
    CONSTRAINT uq_parqueos_bus UNIQUE (id_bus_asignado),
    CONSTRAINT uq_parqueos_estacion_bahia UNIQUE (id_estacion, codigo_bahia),
    CONSTRAINT fk_parqueos_bus FOREIGN KEY (id_bus_asignado) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE
SET NULL,
    CONSTRAINT fk_parqueos_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE RESTRICT
);

-- ---------------------------------------------------------------------
-- 10. EMPLEADOS
--    Datos generales del personal (contacto y direccion cifrados por Laravel).
-- ---------------------------------------------------------------------
CREATE TABLE empleados (
    id_empleado           UUID NOT NULL DEFAULT gen_random_uuid(),
    dpi                   VARCHAR(20) NOT NULL,
    nombre_completo       VARCHAR(200) NOT NULL,
    fecha_nacimiento      DATE NOT NULL,
    rol                   VARCHAR(255) NOT NULL,
    contacto_encriptado   TEXT NOT NULL,
    direccion_encriptada  TEXT NOT NULL,
    activo                BOOLEAN NOT NULL DEFAULT true,
    CONSTRAINT empleados_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT uq_empleados_dpi UNIQUE (dpi),
    CONSTRAINT empleados_rol_check CHECK (rol IN ('piloto', 'seguridad', 'operador'))
);

-- ---------------------------------------------------------------------
-- 11. PILOTOS
--    Punto 9: historial educativo, residencia y comunicacion del piloto.
-- ---------------------------------------------------------------------
CREATE TABLE pilotos (
    id_empleado                    UUID NOT NULL,
    numero_licencia                VARCHAR(30) NOT NULL,
    tipo_licencia                  VARCHAR(10) NOT NULL,
    anios_experiencia              INTEGER NOT NULL DEFAULT 0,
    nivel_educativo                VARCHAR(50) NOT NULL,
    historial_educativo_encriptado  TEXT,
    id_bus_asignado                UUID,
    CONSTRAINT pilotos_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT uq_pilotos_bus UNIQUE (id_bus_asignado),
    CONSTRAINT uq_pilotos_licencia UNIQUE (numero_licencia),
    CONSTRAINT fk_pilotos_bus FOREIGN KEY (id_bus_asignado) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE
SET NULL,
    CONSTRAINT fk_pilotos_empleado FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE ON
DELETE CASCADE,
    CONSTRAINT chk_pilotos_experiencia CHECK (anios_experiencia >= 0)
);

-- ---------------------------------------------------------------------
-- 12. SEGURIDAD
--    Punto 10: al menos un guardia por acceso (trg_seguridad_min_uno).
-- ---------------------------------------------------------------------
CREATE TABLE seguridad (
    id_empleado                    UUID NOT NULL,
    permisos_vigentes              VARCHAR(100) NOT NULL,
    constancia_antecedentes_penales  VARCHAR(100) NOT NULL,
    numero_placa_asignada          VARCHAR(15),
    id_acceso_asignado             UUID,
    CONSTRAINT seguridad_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT fk_seguridad_acceso FOREIGN KEY (id_acceso_asignado) REFERENCES accesos(id_acceso) ON UPDATE CASCADE
ON DELETE SET NULL,
    CONSTRAINT fk_seguridad_empleado FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE
ON DELETE CASCADE
);
```

```
CREATE INDEX idx_seguridad_acceso ON seguridad (id_acceso_asignado);

-- ---------------------------------------------------------------------
-- 13. OPERADORES
--    Punto 16: operador asignado a la PC de trabajo de cada estacion.
-- ---------------------------------------------------------------------
CREATE TABLE operadores (
    id_empleado                 UUID NOT NULL,
    nivel_estudios              VARCHAR(50) NOT NULL,
    anios_experiencia_monitoreo  INTEGER NOT NULL DEFAULT 0,
    id_estacion_asignada        UUID,
    CONSTRAINT operadores_pkey PRIMARY KEY (id_empleado),
    CONSTRAINT fk_operadores_empleado FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON UPDATE CASCADE
ON DELETE CASCADE,
    CONSTRAINT fk_operadores_estacion FOREIGN KEY (id_estacion_asignada) REFERENCES estaciones(id_estacion) ON
UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT chk_operadores_experiencia CHECK (anios_experiencia_monitoreo >= 0)
);
CREATE INDEX idx_operadores_estacion ON operadores (id_estacion_asignada);

-- ---------------------------------------------------------------------
-- 14. TARJETAS
--    Punto 18: Tarjeta Ciudadana (ciudadana, adulto mayor, discapacidad), costo Q20 con 5 viajes gratis.
-- ---------------------------------------------------------------------
CREATE TABLE tarjetas (
    id_tarjeta              UUID NOT NULL DEFAULT gen_random_uuid(),
    numero_tarjeta          VARCHAR(20) NOT NULL,
    saldo                   DECIMAL(8,2) NOT NULL DEFAULT '0'::numeric,
    estado                  VARCHAR(255) NOT NULL DEFAULT 'activa',
    fecha_emision           TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tipo_tarjeta            VARCHAR(20) NOT NULL DEFAULT 'ciudadana',
    nombre_titular          VARCHAR(200),
    dpi_titular             VARCHAR(13),
    costo_emision           DECIMAL(6,2) NOT NULL DEFAULT 20.00,
    viajes_gratis_restantes  INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT tarjetas_pkey PRIMARY KEY (id_tarjeta),
    CONSTRAINT uq_tarjetas_numero UNIQUE (numero_tarjeta),
    CONSTRAINT chk_tarjetas_costo CHECK (costo_emision >= 0),
    CONSTRAINT chk_tarjetas_saldo CHECK (saldo >= 0),
    CONSTRAINT chk_tarjetas_tipo CHECK (tipo_tarjeta IN ('ciudadana', 'adulto_mayor', 'discapacidad')),
    CONSTRAINT chk_tarjetas_titular CHECK (tipo_tarjeta = 'ciudadana' OR (nombre_titular IS NOT NULL AND
dpi_titular IS NOT NULL)),
    CONSTRAINT chk_tarjetas_viajes_gratis CHECK (viajes_gratis_restantes >= 0),
    CONSTRAINT tarjetas_estado_check CHECK (estado IN ('activa', 'bloqueada', 'vencida'))
);

-- ---------------------------------------------------------------------
-- 15. RECARGAS
--    Punto 18: recargas de Q5 a Q200 (efectivo, debito, credito o en linea).
-- ---------------------------------------------------------------------
CREATE TABLE recargas (
    id_recarga   UUID NOT NULL DEFAULT gen_random_uuid(),
    id_tarjeta   UUID NOT NULL,
    monto        DECIMAL(8,2) NOT NULL,
    metodo_pago  VARCHAR(255) NOT NULL,
    fecha_hora   TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT recargas_pkey PRIMARY KEY (id_recarga),
    CONSTRAINT fk_recargas_tarjeta FOREIGN KEY (id_tarjeta) REFERENCES tarjetas(id_tarjeta) ON UPDATE CASCADE ON
DELETE RESTRICT,
    CONSTRAINT chk_recargas_monto CHECK (monto BETWEEN 5 AND 200),
    CONSTRAINT recargas_metodo_pago_check CHECK (metodo_pago IN ('efectivo', 'debito', 'credito', 'en_linea'))
);
CREATE INDEX idx_recargas_tarjeta ON recargas (id_tarjeta);

-- ---------------------------------------------------------------------
-- 16. ALERTAS_SISTEMA
--    Punto 14: alertas de refuerzo de bus (trg_estaciones_alerta_aforo).
-- ---------------------------------------------------------------------
CREATE TABLE alertas_sistema (
    id_alerta       UUID NOT NULL DEFAULT gen_random_uuid(),
    id_estacion     UUID NOT NULL,
    tipo_alerta     VARCHAR(255) NOT NULL,
    descripcion     TEXT NOT NULL,
    estado          VARCHAR(255) NOT NULL DEFAULT 'abierta',
    fecha_creacion  TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_cierre    TIMESTAMP(0),
    CONSTRAINT alertas_sistema_pkey PRIMARY KEY (id_alerta),
    CONSTRAINT fk_alertas_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE CASCADE,
    CONSTRAINT alertas_sistema_estado_check CHECK (estado IN ('abierta', 'atendida', 'cerrada')),
    CONSTRAINT alertas_sistema_tipo_alerta_check CHECK (tipo_alerta IN ('refuerzo_bus', 'bahia_ocupada',
'espera_minima', 'otro'))
);
CREATE INDEX idx_alertas_estado ON alertas_sistema (estado);

-- ---------------------------------------------------------------------
-- 17. TRANSACCIONES_PAGO
--    Punto 18: pagos en molinetes solo con Tarjeta Ciudadana o tarjeta bancaria sin contacto.
-- ---------------------------------------------------------------------
CREATE TABLE transacciones_pago (
    id_transaccion  UUID NOT NULL DEFAULT gen_random_uuid(),
    id_molinete     UUID NOT NULL,
    id_tarjeta      UUID,
    monto           DECIMAL(8,2) NOT NULL,
    metodo_pago     VARCHAR(255) NOT NULL,
    fecha_hora      TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_linea        UUID,
```

```
    viaje_gratis    BOOLEAN NOT NULL DEFAULT false,
    CONSTRAINT transacciones_pago_pkey PRIMARY KEY (id_transaccion),
    CONSTRAINT fk_transacciones_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON
DELETE SET NULL,
    CONSTRAINT fk_transacciones_molinete FOREIGN KEY (id_molinete) REFERENCES molinetes(id_molinete) ON UPDATE
CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_transacciones_tarjeta FOREIGN KEY (id_tarjeta) REFERENCES tarjetas(id_tarjeta) ON UPDATE CASCADE
ON DELETE RESTRICT,
    CONSTRAINT chk_transacciones_monto CHECK (monto >= 0 AND (viaje_gratis OR monto > 0)),
    CONSTRAINT chk_transacciones_tarjeta CHECK (metodo_pago <> 'tarjeta_ciudadana' OR id_tarjeta IS NOT NULL),
    CONSTRAINT transacciones_pago_metodo_pago_check CHECK (metodo_pago IN ('tarjeta_ciudadana',
'tarjeta_bancaria'))
);
CREATE INDEX idx_transacciones_fecha ON transacciones_pago (fecha_hora);
CREATE INDEX idx_transacciones_linea ON transacciones_pago (id_linea);
CREATE INDEX idx_transacciones_tarjeta ON transacciones_pago (id_tarjeta);

-- ---------------------------------------------------------------------
-- 18. BITACORA_AUDITORIA
--    Auditoria de cambios (quien, que tabla, antes y despues).
-- ---------------------------------------------------------------------
CREATE TABLE bitacora_auditoria (
    id_bitacora           UUID NOT NULL DEFAULT gen_random_uuid(),
    id_usuario            BIGINT,
    id_empleado_usuario   UUID,
    tabla_afectada        VARCHAR(100) NOT NULL,
    id_registro_afectado  VARCHAR(36),
    accion                VARCHAR(255) NOT NULL,
    datos_previos         JSONB,
    datos_nuevos          JSONB,
    fecha_registro        TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT bitacora_auditoria_pkey PRIMARY KEY (id_bitacora),
    CONSTRAINT fk_bitacora_empleado FOREIGN KEY (id_empleado_usuario) REFERENCES empleados(id_empleado) ON UPDATE
CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_bitacora_usuario FOREIGN KEY (id_usuario) REFERENCES users(id) ON UPDATE CASCADE ON DELETE SET
NULL,
    CONSTRAINT bitacora_auditoria_accion_check CHECK (accion IN ('INSERT', 'UPDATE', 'DELETE'))
);
CREATE INDEX idx_bitacora_tabla ON bitacora_auditoria (tabla_afectada, id_registro_afectado);

-- ---------------------------------------------------------------------
-- 19. VIAJES
--    Punto 1: recorridos reales de cada bus.
-- ---------------------------------------------------------------------
CREATE TABLE viajes (
    id_viaje           UUID NOT NULL DEFAULT gen_random_uuid(),
    id_bus             UUID NOT NULL,
    id_linea           UUID NOT NULL,
    fecha_hora_inicio  TIMESTAMP(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_hora_fin     TIMESTAMP(0),
    estado             VARCHAR(255) NOT NULL DEFAULT 'en_curso',
    CONSTRAINT viajes_pkey PRIMARY KEY (id_viaje),
    CONSTRAINT fk_viajes_bus FOREIGN KEY (id_bus) REFERENCES buses(id_bus) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_viajes_linea FOREIGN KEY (id_linea) REFERENCES lineas(id_linea) ON UPDATE CASCADE ON DELETE
RESTRICT,
    CONSTRAINT chk_viajes_fechas CHECK (fecha_hora_fin IS NULL OR fecha_hora_fin >= fecha_hora_inicio),
    CONSTRAINT viajes_estado_check CHECK (estado IN ('en_curso', 'finalizado', 'cancelado'))
);
CREATE INDEX idx_viajes_bus ON viajes (id_bus);
CREATE INDEX idx_viajes_linea ON viajes (id_linea);

-- ---------------------------------------------------------------------
-- 20. VISITAS_VIAJE
--    Puntos 1 y 15: paradas visitadas con hora de llegada/salida y ocupacion.
-- ---------------------------------------------------------------------
CREATE TABLE visitas_viaje (
    id_visita                UUID NOT NULL DEFAULT gen_random_uuid(),
    id_viaje                 UUID NOT NULL,
    id_estacion              UUID NOT NULL,
    orden_visita             INTEGER NOT NULL,
    hora_llegada             TIMESTAMP(0),
    hora_salida              TIMESTAMP(0),
    ocupacion_al_llegar_pct  DECIMAL(5,2),
    CONSTRAINT visitas_viaje_pkey PRIMARY KEY (id_visita),
    CONSTRAINT uq_visitas_viaje_orden UNIQUE (id_viaje, orden_visita),
    CONSTRAINT fk_visitas_estacion FOREIGN KEY (id_estacion) REFERENCES estaciones(id_estacion) ON UPDATE CASCADE
ON DELETE RESTRICT,
    CONSTRAINT fk_visitas_viaje FOREIGN KEY (id_viaje) REFERENCES viajes(id_viaje) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT chk_visitas_horas CHECK (hora_salida IS NULL OR hora_llegada IS NULL OR hora_salida >=
hora_llegada),
    CONSTRAINT chk_visitas_ocupacion CHECK (ocupacion_al_llegar_pct IS NULL OR ocupacion_al_llegar_pct BETWEEN 0
AND 100),
    CONSTRAINT chk_visitas_orden CHECK (orden_visita >= 1)
);
CREATE INDEX idx_visitas_estacion ON visitas_viaje (id_estacion, hora_llegada);

-- =====================================================================
-- USUARIOS DEL SISTEMA (tabla users de Laravel): rol y empleado vinculado
-- =====================================================================
ALTER TABLE users
    ADD COLUMN rol VARCHAR(255) NOT NULL DEFAULT 'operador' CHECK (rol IN ('administrador','operador')),
    ADD COLUMN id_empleado UUID NULL,
    ADD CONSTRAINT uq_users_empleado UNIQUE (id_empleado),
    ADD CONSTRAINT fk_users_empleado FOREIGN KEY (id_empleado) REFERENCES empleados (id_empleado) ON UPDATE CASCADE
ON DELETE SET NULL;
```

```
-- ========================================================================
-- REGLAS DE NEGOCIO: funciones, triggers y procedimiento almacenado
-- ========================================================================
-- Punto 3 (regla propia del equipo): una estación debe conservar al menos 2 accesos.
CREATE OR REPLACE FUNCTION fn_accesos_min_dos() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_conteo INT;
BEGIN
    SELECT COUNT(*) INTO v_conteo FROM accesos WHERE id_estacion = OLD.id_estacion;
    IF v_conteo <= 2 THEN
        RAISE EXCEPTION 'No se puede eliminar: la estacion debe conservar al menos 2 accesos.';
    END IF;
    RETURN OLD;
END;
$$;

CREATE TRIGGER trg_accesos_min_dos_before_delete
BEFORE DELETE ON accesos
FOR EACH ROW EXECUTE FUNCTION fn_accesos_min_dos();

-- Punto 10: ningún acceso puede quedarse sin guardia al reasignar o dar de baja al último.
CREATE OR REPLACE FUNCTION fn_seguridad_min_uno() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_conteo INT;
BEGIN
    IF OLD.id_acceso_asignado IS NOT NULL
       AND (TG_OP = 'DELETE' OR NEW.id_acceso_asignado IS DISTINCT FROM OLD.id_acceso_asignado)
       AND EXISTS (SELECT 1 FROM accesos WHERE id_acceso = OLD.id_acceso_asignado) THEN
        SELECT COUNT(*) INTO v_conteo FROM seguridad WHERE id_acceso_asignado = OLD.id_acceso_asignado;
        IF v_conteo <= 1 THEN
            RAISE EXCEPTION 'No se puede reasignar ni dar de baja: el acceso quedaria sin personal de seguridad.';
        END IF;
    END IF;
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_seguridad_min_uno
BEFORE UPDATE OR DELETE ON seguridad
FOR EACH ROW EXECUTE FUNCTION fn_seguridad_min_uno();

-- Punto 5: una línea tiene como máximo el doble de buses que de estaciones.
CREATE OR REPLACE FUNCTION fn_buses_max_flota() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_buses_actuales INT;
    v_paradas INT;
BEGIN
    IF NEW.id_linea_asignada IS NOT NULL
       AND (TG_OP = 'INSERT' OR NEW.id_linea_asignada IS DISTINCT FROM OLD.id_linea_asignada) THEN
        SELECT COUNT(*) INTO v_buses_actuales FROM buses WHERE id_linea_asignada = NEW.id_linea_asignada;
        SELECT COUNT(*) INTO v_paradas FROM linea_estacion WHERE id_linea = NEW.id_linea_asignada;
        IF v_buses_actuales >= 2 * v_paradas THEN
            RAISE EXCEPTION 'No se puede asignar el bus: se alcanzo el maximo permitido (2x el numero de paradas)
para esta linea.';
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_buses_max_flota
BEFORE INSERT OR UPDATE OF id_linea_asignada ON buses
FOR EACH ROW EXECUTE FUNCTION fn_buses_max_flota();

-- Punto 14: alerta automática cuando la estación supera en 50 % su aforo (150 %).
CREATE OR REPLACE FUNCTION fn_estaciones_alerta_aforo() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.aforo_actual >= 1.5 * NEW.aforo_maximo
       AND OLD.aforo_actual < 1.5 * OLD.aforo_maximo THEN
        INSERT INTO alertas_sistema (id_estacion, tipo_alerta, descripcion, estado)
        VALUES (
            NEW.id_estacion,
            'refuerzo_bus',
            'La estacion supero el 150% de su aforo maximo (' || NEW.aforo_actual || '/' || NEW.aforo_maximo
                || '). Se requiere unidad de refuerzo.',
            'abierta'
        );
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_estaciones_alerta_aforo
AFTER UPDATE OF aforo_actual, aforo_maximo ON estaciones
FOR EACH ROW EXECUTE FUNCTION fn_estaciones_alerta_aforo();

-- Punto 13: la distancia total de la línea es la suma de las distancias entre sus paradas.
CREATE OR REPLACE FUNCTION fn_linea_estacion_distancia() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
```

```
    IF TG_OP IN ('UPDATE', 'DELETE') THEN
        UPDATE lineas
           SET distancia_total_km = COALESCE((SELECT SUM(distancia_previa_km) FROM linea_estacion WHERE id_linea =
OLD.id_linea), 0)
         WHERE id_linea = OLD.id_linea;
    END IF;
    IF TG_OP IN ('INSERT', 'UPDATE') THEN
        UPDATE lineas
           SET distancia_total_km = COALESCE((SELECT SUM(distancia_previa_km) FROM linea_estacion WHERE id_linea =
NEW.id_linea), 0)
         WHERE id_linea = NEW.id_linea;
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_linea_estacion_distancia
AFTER INSERT OR UPDATE OR DELETE ON linea_estacion
FOR EACH ROW EXECUTE FUNCTION fn_linea_estacion_distancia();

-- Punto 7: cambio de parqueo en una sola operación; el bus nunca queda sin parqueo.
-- Primero libera el parqueo actual y luego ocupa el nuevo (respetando el UNIQUE).
CREATE OR REPLACE PROCEDURE sp_cambiar_parqueo_bus(p_id_bus UUID, p_id_parqueo_nuevo UUID)
LANGUAGE plpgsql AS $$
DECLARE
    v_parqueo_actual UUID;
    v_ocupado UUID;
BEGIN
    SELECT id_bus_asignado INTO v_ocupado
      FROM parqueos WHERE id_parqueo = p_id_parqueo_nuevo
       FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'El parqueo destino no existe.';
    END IF;
    IF v_ocupado = p_id_bus THEN
        RETURN;
    END IF;
    IF v_ocupado IS NOT NULL THEN
        RAISE EXCEPTION 'El parqueo destino ya esta ocupado por otro bus.';
    END IF;

    SELECT id_parqueo INTO v_parqueo_actual
      FROM parqueos WHERE id_bus_asignado = p_id_bus
       FOR UPDATE;
    IF v_parqueo_actual IS NOT NULL THEN
        UPDATE parqueos SET id_bus_asignado = NULL WHERE id_parqueo = v_parqueo_actual;
    END IF;
    UPDATE parqueos SET id_bus_asignado = p_id_bus WHERE id_parqueo = p_id_parqueo_nuevo;
END;
$$;

-- ======================================================================
-- SEGURIDAD: Row Level Security en todas las tablas del sistema
-- ======================================================================
ALTER TABLE municipios ENABLE ROW LEVEL SECURITY;
ALTER TABLE lineas ENABLE ROW LEVEL SECURITY;
ALTER TABLE estaciones ENABLE ROW LEVEL SECURITY;
ALTER TABLE linea_estacion ENABLE ROW LEVEL SECURITY;
ALTER TABLE horarios_linea ENABLE ROW LEVEL SECURITY;
ALTER TABLE buses ENABLE ROW LEVEL SECURITY;
ALTER TABLE accesos ENABLE ROW LEVEL SECURITY;
ALTER TABLE molinetes ENABLE ROW LEVEL SECURITY;
ALTER TABLE parqueos ENABLE ROW LEVEL SECURITY;
ALTER TABLE empleados ENABLE ROW LEVEL SECURITY;
ALTER TABLE pilotos ENABLE ROW LEVEL SECURITY;
ALTER TABLE seguridad ENABLE ROW LEVEL SECURITY;
ALTER TABLE operadores ENABLE ROW LEVEL SECURITY;
ALTER TABLE tarjetas ENABLE ROW LEVEL SECURITY;
ALTER TABLE recargas ENABLE ROW LEVEL SECURITY;
ALTER TABLE alertas_sistema ENABLE ROW LEVEL SECURITY;
ALTER TABLE transacciones_pago ENABLE ROW LEVEL SECURITY;
ALTER TABLE bitacora_auditoria ENABLE ROW LEVEL SECURITY;
ALTER TABLE viajes ENABLE ROW LEVEL SECURITY;
ALTER TABLE visitas_viaje ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
```

9.2. Consultas SQL de los reportes

El enunciado pide dejar constancia de las consultas de los reportes. Estas son las consultas que ejecuta la aplicación (punto 15); los parámetros :linea y :municipio son filtros opcionales.

```
-- Sistema de Control Transmetro — Consultas SQL de los reportes (punto 15)
-- PostgreSQL. Los parámetros :linea y :municipio son filtros opcionales (NULL = todos).
-- Este archivo se genera con: php artisan transmetro:consultas-reportes

-- ======================================================================
-- Reporte de estaciones (Puntos 2, 3, 6, 10, 11 y 14)
-- Todas las estaciones con su municipio, las líneas que pasan por ellas, aforo, accesos, guardias y parqueos.
-- ======================================================================
SELECT e.nombre AS estacion,
       m.nombre AS municipio,
       e.direccion,
```

```
       string_agg(l.codigo_linea, ', ' ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'),
coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '', 'g'), '')::int, 0)) AS lineas,
       e.aforo_maximo,
       e.aforo_actual,
       round(e.aforo_actual * 100.0 / e.aforo_maximo, 1) AS ocupacion_pct,
       CASE WHEN e.aforo_actual >= 1.5 * e.aforo_maximo THEN 'Requiere refuerzo' ELSE 'Normal' END AS estado_aforo,
       (SELECT count(*) FROM accesos a WHERE a.id_estacion = e.id_estacion) AS accesos,
       (SELECT count(*) FROM seguridad s
          JOIN accesos a ON a.id_acceso = s.id_acceso_asignado
         WHERE a.id_estacion = e.id_estacion) AS guardias,
       (SELECT count(*) FROM parqueos p WHERE p.id_estacion = e.id_estacion) AS parqueos
  FROM estaciones e
  JOIN municipios m ON m.id_municipio = e.id_municipio
  LEFT JOIN linea_estacion le ON le.id_estacion = e.id_estacion
  LEFT JOIN lineas l ON l.id_linea = le.id_linea
 WHERE (CAST(:linea AS varchar) IS NULL
        OR e.id_estacion IN (SELECT x.id_estacion
                               FROM linea_estacion x
                               JOIN lineas y ON y.id_linea = x.id_linea
                              WHERE y.codigo_linea = :linea))
   AND (CAST(:municipio AS varchar) IS NULL OR m.nombre = :municipio)
 GROUP BY e.id_estacion, m.nombre
 ORDER BY e.nombre;

-- ========================================================================
-- Reporte de líneas (Puntos 4, 5, 8, 11 y 13)
-- Cada línea con su municipio, número de estaciones, distancia total, accesos y buses asignados, y si cumple la
regla de n a 2n buses.
-- ========================================================================
SELECT l.codigo_linea AS codigo,
       l.nombre,
       m.nombre AS municipio,
       count(le.id_estacion) AS estaciones,
       l.distancia_total_km,
       (SELECT count(*) FROM accesos a
          JOIN linea_estacion x ON x.id_estacion = a.id_estacion
         WHERE x.id_linea = l.id_linea) AS accesos,
       (SELECT count(*) FROM buses b WHERE b.id_linea_asignada = l.id_linea) AS buses_asignados,
       count(le.id_estacion) AS minimo_buses,
       2 * count(le.id_estacion) AS maximo_buses,
       CASE WHEN (SELECT count(*) FROM buses b WHERE b.id_linea_asignada = l.id_linea)
                 BETWEEN count(le.id_estacion) AND 2 * count(le.id_estacion)
            THEN 'Cumple' ELSE 'No cumple' END AS regla_buses,
       l.tarifa,
       l.estado
  FROM lineas l
  JOIN municipios m ON m.id_municipio = l.id_municipio
  LEFT JOIN linea_estacion le ON le.id_linea = l.id_linea
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
   AND (CAST(:municipio AS varchar) IS NULL OR m.nombre = :municipio)
 GROUP BY l.id_linea, m.nombre
 ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'), coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '',
'g'), '')::int, 0);

-- ========================================================================
-- Buses asignados a cada línea (Puntos 4, 6, 7, 9 y 15)
-- Buses de cada línea con su capacidad, ocupación, parqueo y piloto; al final, los buses sin línea asignada.
-- ========================================================================
SELECT coalesce(l.codigo_linea, 'Sin línea') AS linea,
       b.placa,
       b.capacidad_maxima,
       b.ocupacion_actual_pct,
       CASE WHEN l.id_linea IS NOT NULL AND b.ocupacion_actual_pct < 25
            THEN '+5 min por estación' ELSE '—' END AS espera_extra,
       b.estado_logico,
       e.nombre || ' · ' || p.codigo_bahia AS parqueo,
       emp.nombre_completo AS piloto,
       pi.tipo_licencia || ' ' || pi.numero_licencia AS licencia
  FROM buses b
  LEFT JOIN lineas l ON l.id_linea = b.id_linea_asignada
  LEFT JOIN parqueos p ON p.id_bus_asignado = b.id_bus
  LEFT JOIN estaciones e ON e.id_estacion = p.id_estacion
  LEFT JOIN pilotos pi ON pi.id_bus_asignado = b.id_bus
  LEFT JOIN empleados emp ON emp.id_empleado = pi.id_empleado
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY (l.id_linea IS NULL), substring(l.codigo_linea from '^[A-Za-z]*'),
coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '', 'g'), '')::int, 0), b.placa;

-- ========================================================================
-- Recorrido y distancias de cada línea (Puntos 1, 12 y 13)
-- Estaciones de cada línea en el orden de la ruta, con la distancia desde la parada anterior y la distancia
acumulada.
-- ========================================================================
SELECT l.codigo_linea AS linea,
       le.orden_parada,
       e.nombre AS estacion,
       e.direccion,
       le.distancia_previa_km,
       sum(le.distancia_previa_km) OVER (PARTITION BY l.id_linea ORDER BY le.orden_parada) AS km_acumulado
  FROM linea_estacion le
  JOIN lineas l ON l.id_linea = le.id_linea
  JOIN estaciones e ON e.id_estacion = le.id_estacion
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY substring(l.codigo_linea from '^[A-Za-z]*'), coalesce(nullif(regexp_replace(l.codigo_linea, '[^0-9]', '',
'g'), '')::int, 0), le.orden_parada;

-- ========================================================================
```

```
-- Viajes y visitas de las unidades (Puntos 1 y 15)
-- Estaciones que visitó cada bus en sus viajes: hora de llegada y salida, minutos detenido y ocupación al llegar
(últimos 500 registros).
-- ========================================================================
SELECT l.codigo_linea AS linea,
       b.placa AS bus,
       v.fecha_hora_inicio AS inicio_viaje,
       v.estado AS estado_viaje,
       vv.orden_visita,
       e.nombre AS estacion,
       vv.hora_llegada,
       vv.hora_salida,
       round(extract(epoch FROM (vv.hora_salida - vv.hora_llegada)) / 60, 1) AS minutos_detenido,
       vv.ocupacion_al_llegar_pct
  FROM visitas_viaje vv
  JOIN viajes v ON v.id_viaje = vv.id_viaje
  JOIN buses b ON b.id_bus = v.id_bus
  JOIN lineas l ON l.id_linea = v.id_linea
  JOIN estaciones e ON e.id_estacion = vv.id_estacion
 WHERE (CAST(:linea AS varchar) IS NULL OR l.codigo_linea = :linea)
 ORDER BY v.fecha_hora_inicio DESC, b.placa, vv.orden_visita
 LIMIT 500;
```

10. Diagramas UML

10.1. Diagrama de Casos de Uso

Representa las interacciones entre los actores del sistema (Administrador/Municipalidad, Operador de Estación, Pasajero y el propio Sistema de forma automática) y las funcionalidades que cada uno puede ejecutar, agrupadas por módulo. Los casos de uso del piloto y del personal de seguridad se muestran como fase 2.

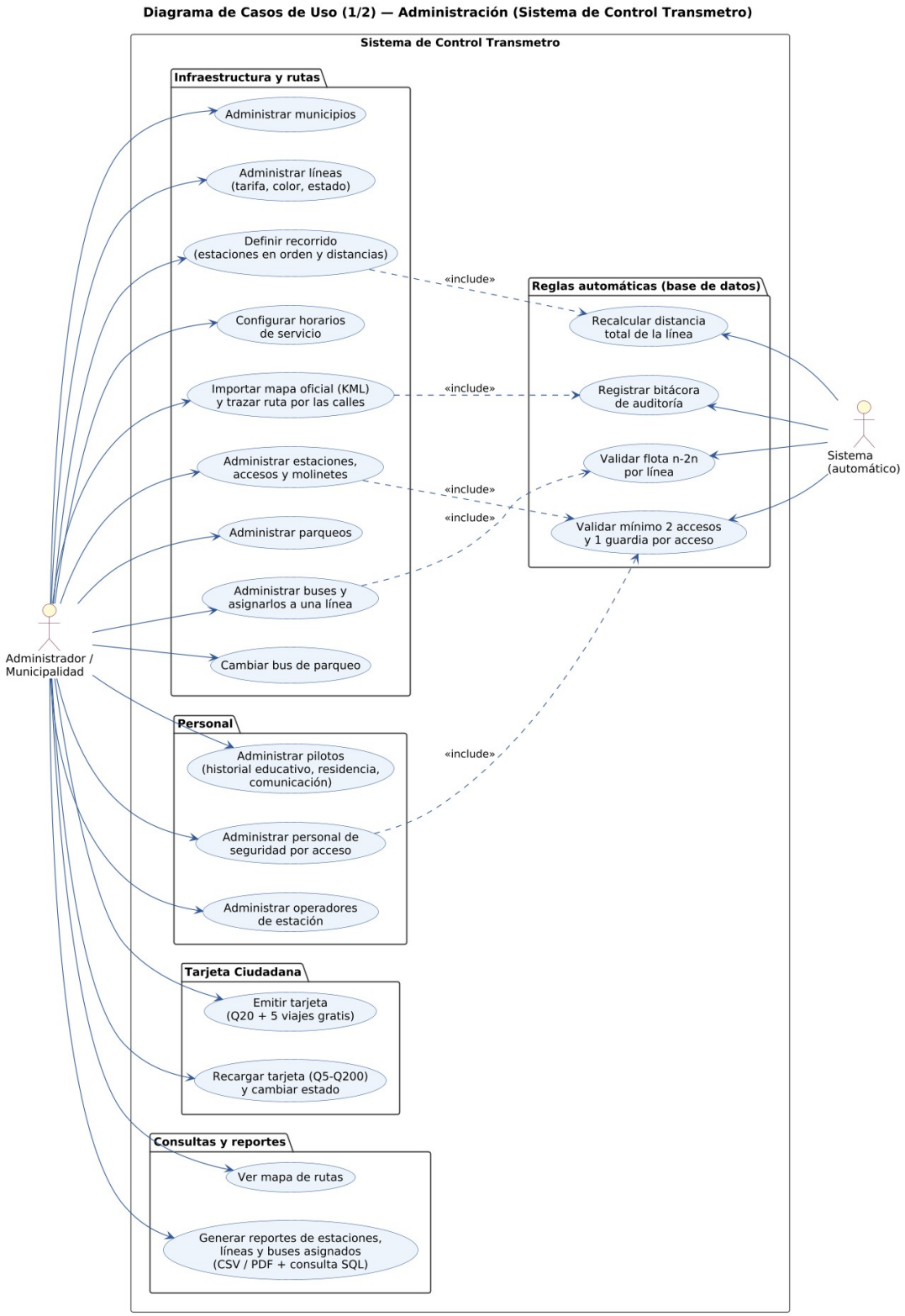


Figura 2. Casos de uso (1/2) — Administración de infraestructura, personal, tarjetas y reportes.

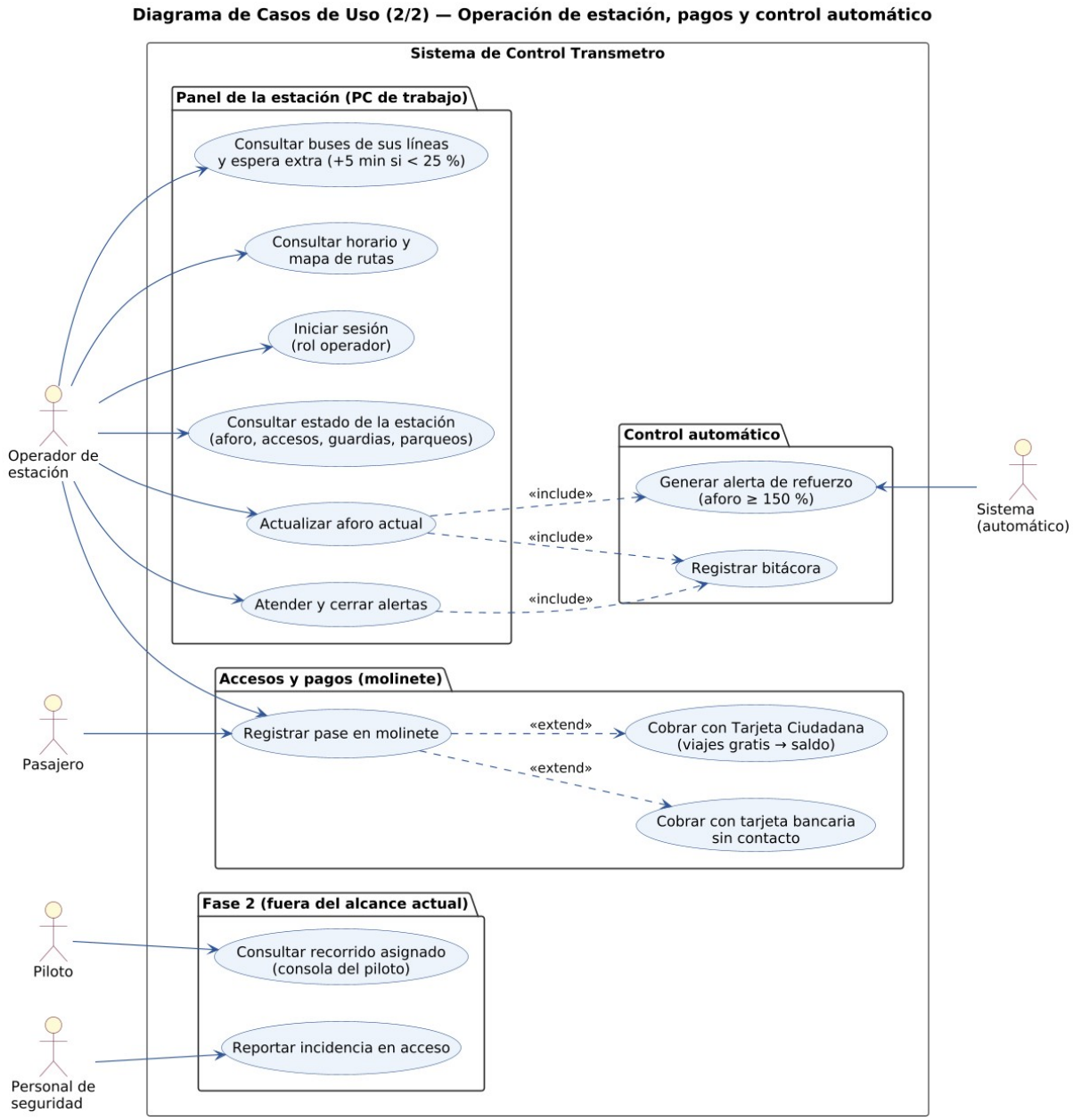


Figura 3. Casos de uso (2/2) — Operación de estación, pagos y control automático.

10.2. Diagrama de Componentes

Muestra la organización física y lógica de la solución: el navegador de la PC de cada estación se comunica por HTTPS con la aplicación Laravel alojada en Render, y solo la aplicación se conecta a la base de datos PostgreSQL en Supabase. El mapa usa teselas de OpenStreetMap y el servicio OSRM desde el navegador, y GitHub Actions ejecuta las pruebas en cada cambio (ver RNF-11 y la sección 6.1).

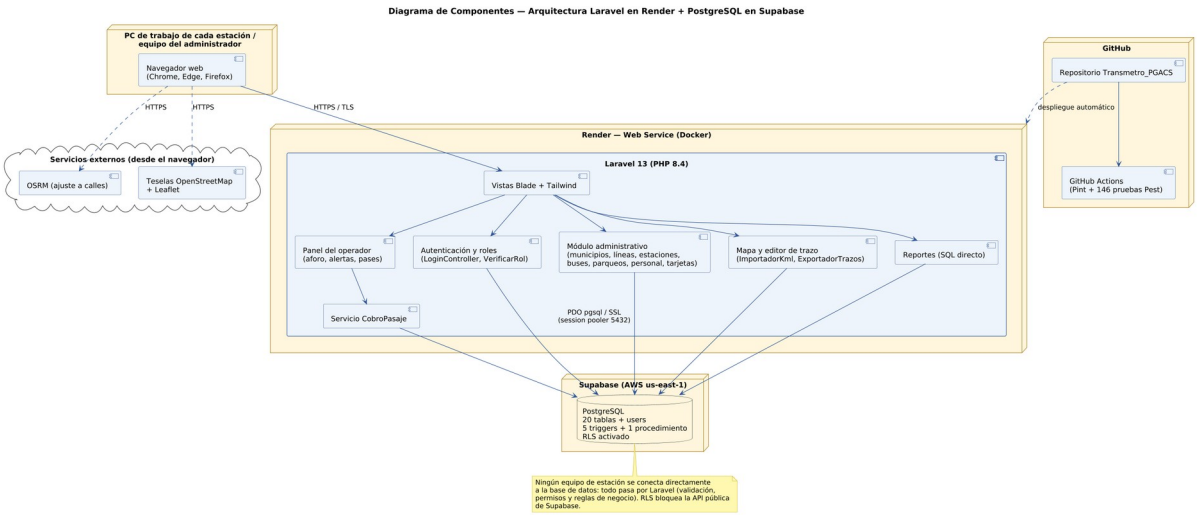


Figura 4. Diagrama de componentes de la arquitectura del sistema.

10.3. Diagrama de Clases

Traduce a nivel orientado a objetos el Modelo Entidad Relación, incluyendo la generalización de Empleado en Piloto, Seguridad y Operador, las clases agregadas durante el desarrollo (HorarioLinea, Tarjeta, Recarga y Usuario), el servicio CobroPasaje y las multiplicidades tal como están implementadas en las llaves foráneas y restricciones del esquema.

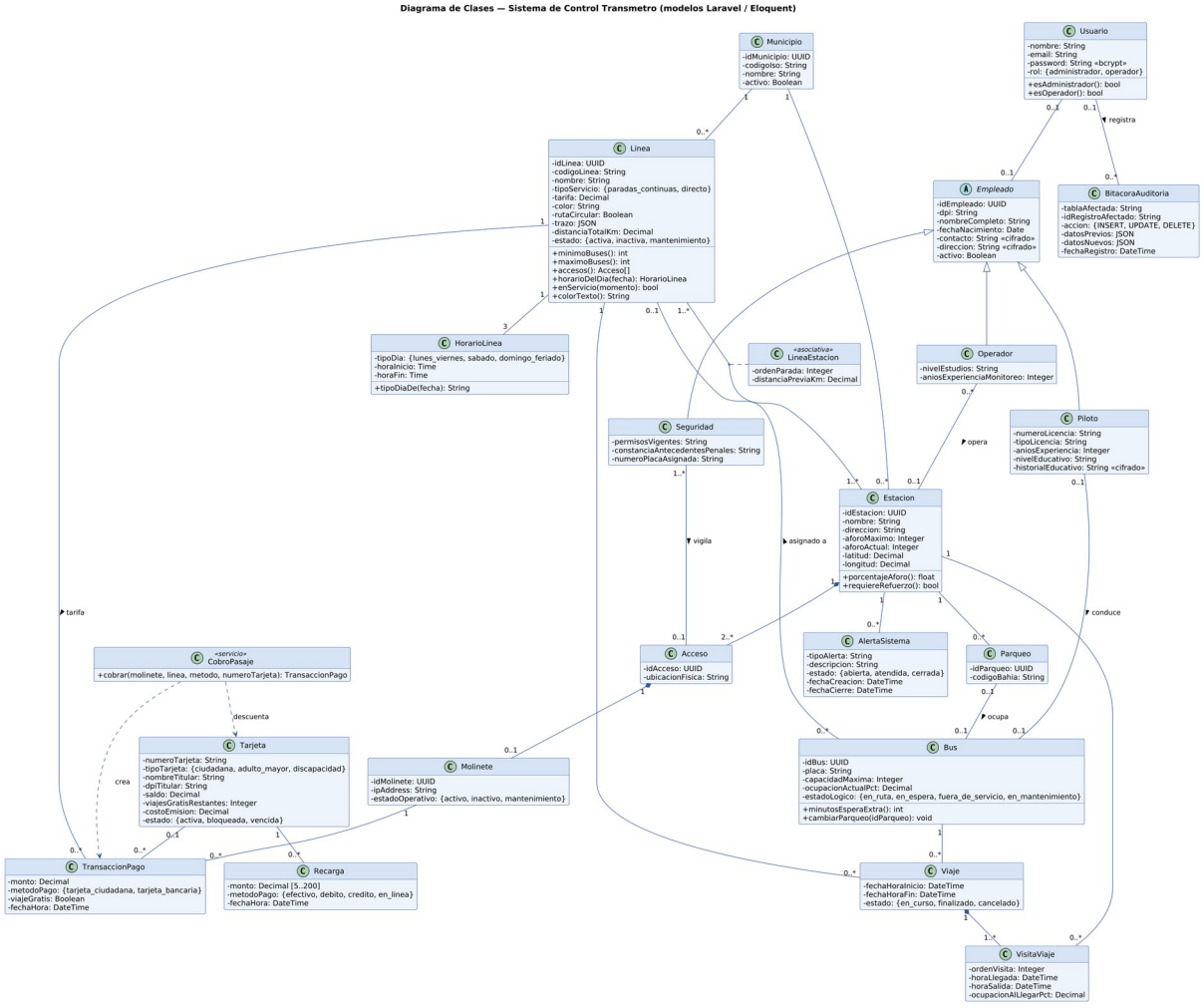


Figura 5. Diagrama de clases del Sistema de Control Transmetro.

10.4. Diagrama de Objetos

Presenta una instancia concreta del diagrama de clases con datos reales cargados en el sistema: el transbordo en la estación San Sebastián entre las Líneas 1 y 2, el bus TM-001 con baja ocupación, un acceso con su guardia y molinete, un pase con viaje gratis de la Tarjeta Ciudadana y una alerta de refuerzo abierta porque la estación supera el 150 % de su aforo.

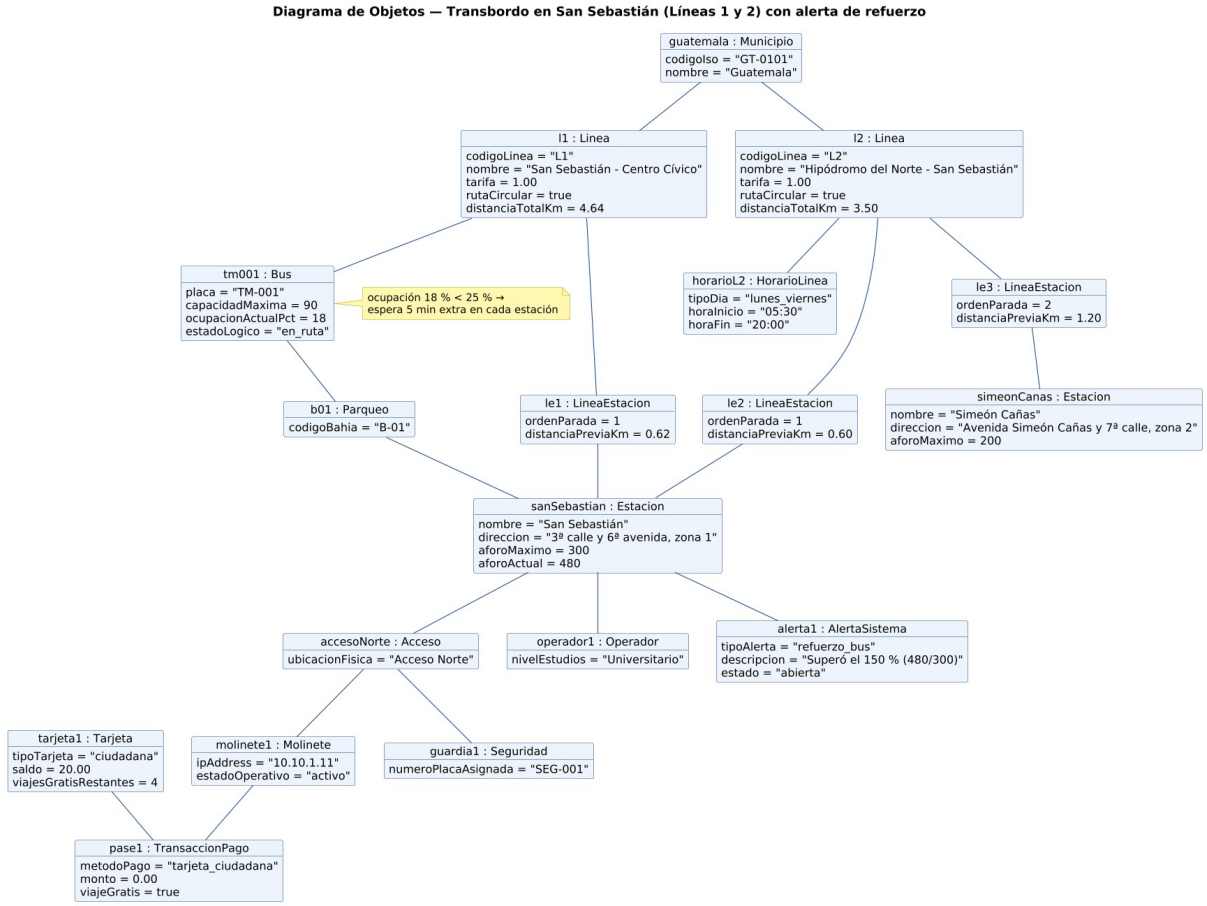


Figura 6. Diagrama de objetos — transbordo en San Sebastián (Líneas 1 y 2).

10.5. Diagrama de Secuencia

Detalla la interacción entre el pasajero, el operador de estación, el panel web, el controlador de Laravel, el servicio CobroPasaje y la base de datos durante el registro de un pase con Tarjeta Ciudadana (primero los viajes gratis y después el saldo), y la actualización del aforo que genera automáticamente la alerta de refuerzo.

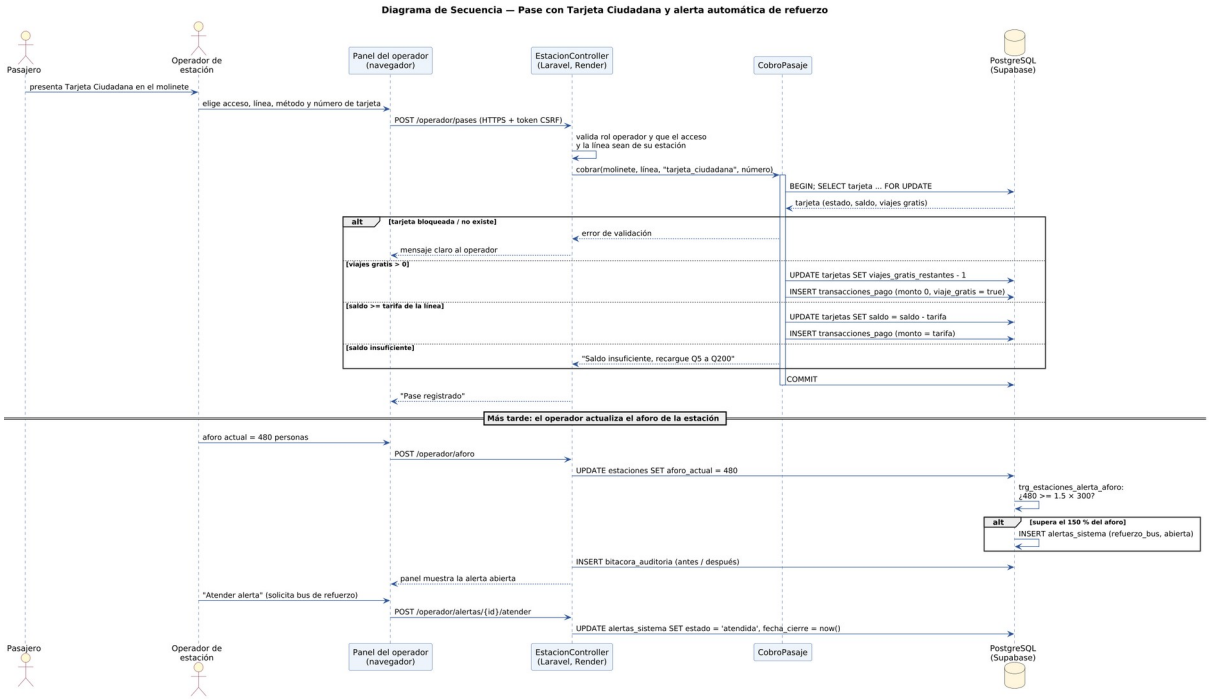


Figura 7. Secuencia de registro de un pase y alerta automática de refuerzo.

10.6. Diagrama de Actividades

Describe, mediante carriles (swimlanes) de Administrador y Sistema (Backend), el flujo completo de asignación de un bus a una línea: la validación del límite de flota, el rechazo de asignaciones duplicadas, la verificación del parqueo obligatorio y el registro de la operación en la bitácora de auditoría.

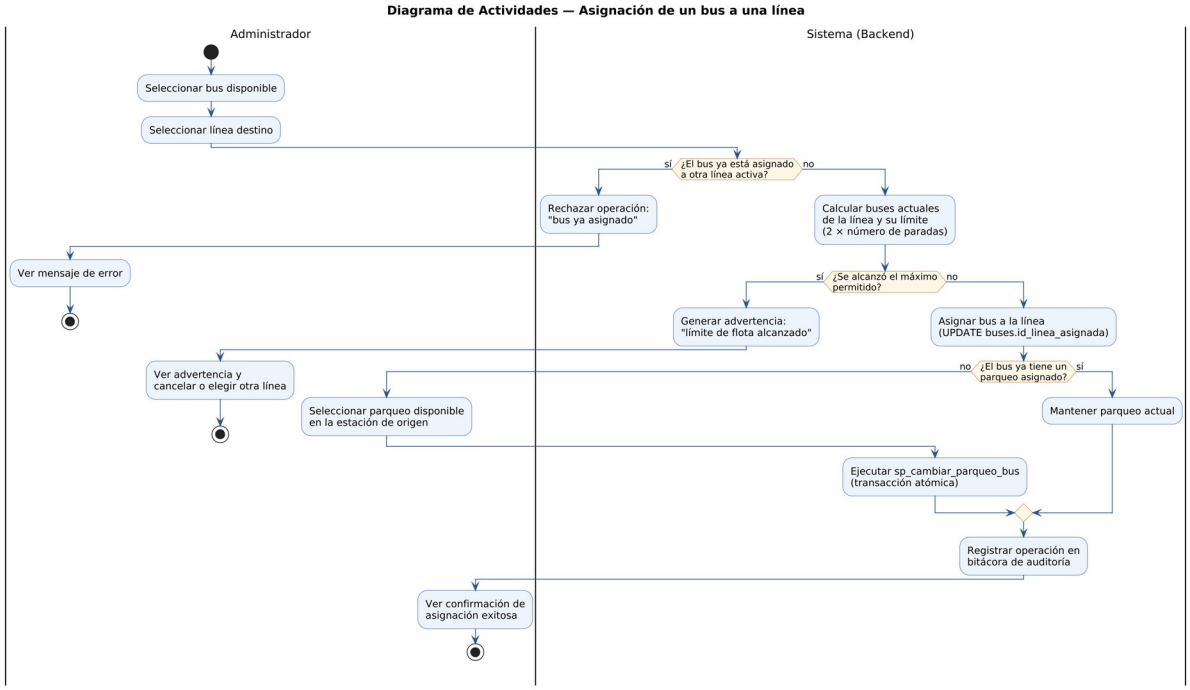


Figura 8. Actividades del proceso de asignación de un bus a una línea.

11. Presentación de Stack Tecnológico

El stack tecnológico es el conjunto de herramientas con las que se construye, prueba y opera el sistema. Se eligieron herramientas de código abierto, con documentación amplia y sin costo de licencias, que cumplen los requerimientos de una aplicación web alojada en la nube (puntos 12, 13 y 14 del enunciado).

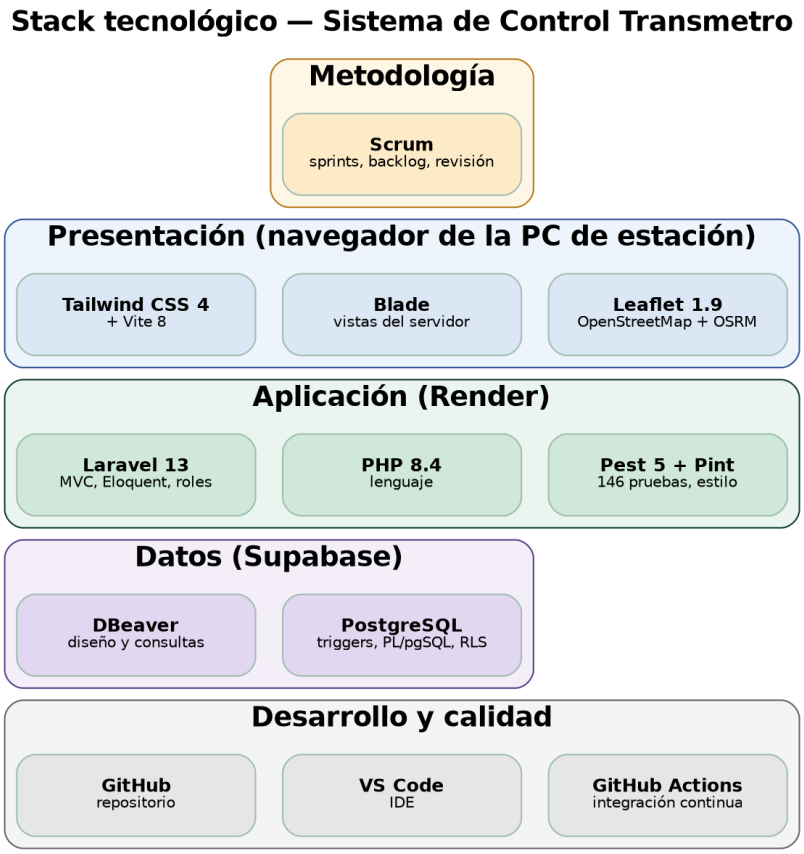


Figura 9. Stack tecnológico por capas.

11.1. Componentes principales

Categoría	Herramienta	Versión	Por qué se eligió
Base de datos	PostgreSQL (Supabase en la nube)	16 local / 15+ en Supabase	Integridad referencial, triggers y procedimientos en PL/pgSQL, restricciones CHECK, JSONB para el trazo de las rutas y Row Level Security. Supabase ofrece PostgreSQL administrado, conexión SSL y respaldos diarios en el plan Pro.
Metodología	Scrum	—	Entregas incrementales por sprint, backlog priorizado según los puntos del enunciado y revisión frecuente del avance.
Lenguaje	PHP	8.4	Lenguaje maduro para la web, con tipado moderno, amplio soporte de alojamiento y gran comunidad. Se complementa con SQL (PL/pgSQL) para las reglas de negocio y JavaScript para el mapa.
Framework	Laravel	13	Arquitectura MVC, ORM Eloquent, migraciones versionadas, protección CSRF, contraseñas con bcrypt, cifrado de datos, middleware de roles y vistas Blade.
IDE	Visual Studio Code	Última estable	Gratuito y liviano, con extensiones para PHP, Laravel, PostgreSQL y Git, y terminal integrada.

11.2. Herramientas complementarias

Herramienta	Uso en el proyecto
DBeaver	Diseño, consulta y administración de la base de datos
Git y GitHub	Control de versiones y repositorio del proyecto
GitHub Actions	Integración continua: revisa el estilo con Pint y ejecuta las 146 pruebas en cada cambio
Pest 5	Pruebas automatizadas unitarias y de funcionalidad
Laravel Pint	Formato de código según el estándar PSR-12
Tailwind CSS 4 y Vite 8	Estilos de la interfaz y compilación de recursos
Leaflet 1.9 y OpenStreetMap	Mapa de líneas y estaciones
OSRM	Ajuste del trazo de las rutas a las calles
Render	Alojamiento de la aplicación (servicio web con Docker)
Supabase	Alojamiento de la base de datos PostgreSQL
PlantUML y Graphviz	Diagramas UML y modelo entidad relación
SonarQube Cloud (por configurar)	Análisis estático de calidad: errores, vulnerabilidades, duplicación y deuda técnica

11.3. Aplicación de Scrum en el proyecto

- **Product Owner:** representante de la municipalidad; define y prioriza el backlog a partir de los puntos del enunciado.
- **Scrum Master y equipo de desarrollo:** la consultora; facilita los sprints, desarrolla, prueba y documenta.

- **Artefactos:** backlog del producto (requerimientos RF y RNF), backlog de cada sprint e incremento funcional con pruebas aprobadas.
- **Eventos:** planificación del sprint, seguimiento diario, revisión con demostración del sistema y retrospectiva.

Incremento	Contenido	Estado
1	Modelo de datos, migraciones, triggers, procedimiento almacenado y seguridad (RLS)	Terminado
2	Autenticación por roles, módulo administrativo y panel del operador	Terminado
3	Tarjeta Ciudadana, recargas y cobro en molinete	Terminado
4	Líneas oficiales, horarios, mapa, importación KML y trazo por las calles	Terminado
5	Reportes con consulta SQL y exportación CSV; pruebas automatizadas e integración continua	Terminado
6	Bitácora en pantalla, reportes de alertas, transacciones y personal, SonarQube Cloud y despliegue en Render	Pendiente

12. Métricas de Software

Las métricas de software permiten medir el tamaño, la complejidad, la calidad y el avance del sistema con datos objetivos. Los valores se obtuvieron del código del repositorio y de la ejecución de las pruebas al 30 de septiembre de 2026; al final se indican las métricas que faltan y la herramienta para obtenerlas.

12.1. Métricas de tamaño

Capa	Archivos	Líneas físicas	Líneas de código	Líneas de comentario
Lógica de la aplicación (app/)	41	3,426	1,893	283
Migraciones (estructura y reglas de BD)	23	1,233	799	132
Datos iniciales (seeders y mapas)	5	755	490	89
Rutas	2	101	70	7
Vistas Blade	32	2,042	2,042	—
Total de código de producción	**103**	**7,557**	**5,294**	**511**
Pruebas automatizadas (tests/)	15	1,203	716	91

Además: 41 clases y 204 métodos en app/ (5.0 métodos por clase), densidad de comentarios del 13 % en la lógica de la aplicación, script SQL de 565 líneas y archivo de consultas de reportes de 123 líneas.

12.2. Métricas de complejidad

Se midió la complejidad ciclomática de McCabe de cada método (1 + número de decisiones: if, for, foreach, while, case, catch, &&, ||, ?? y ?).

Indicador	app/	Migraciones	Seeders	Pruebas
Complejidad ciclomática promedio por método	1.96	1.04	7.79	1.11
Complejidad máxima	18	2	60	3
Métodos con complejidad mayor a 10	3	0	2	0

Métodos que superan el límite recomendado de 10 y que conviene refactorizar: ImportadorKml::encadenar (18), ImportadorKml::leer (14), TrazoLineaController::puntos (13), TransmetroDatos::completarLinea (11) y TransmetroDatos::generar (60, porque concentra la carga de los datos iniciales). Ninguno pertenece a la operación diaria del operador.

12.3. Métricas de la base de datos

Indicador	Valor
Tablas del dominio + users	20 + 1
Columnas	135
Llaves foráneas	28
Restricciones CHECK	39
Índices	65
Triggers / procedimientos almacenados	5 / 1
Tablas con Row Level Security	21 de 21 (100 %)
Reglas del enunciado validadas en la base de datos	6 de 6 reglas de integridad (puntos 3, 5, 7, 10, 13 y 14)

12.4. Métricas de calidad: pruebas

Archivo de pruebas	Pruebas	Aserciones	Qué verifica
ReglasModelosTest (unitarias)	20	20	Reglas de los modelos: flota n-2n, espera extra, porcentajes
AutenticacionTest	12	36	Inicio de sesión y acceso por rol
PantallasTest	17	25	Que cada pantalla cargue para el rol correcto
ReglasBaseDatosTest	12	23	Triggers y procedimiento almacenado
CobroPasajeTest	8	31	Viajes gratis, saldo, tarjeta bloqueada y tarjeta bancaria
TarjetasTest	7	21	Emisión, recargas de Q5 a Q200 y bloqueo
OperadorTest	7	20	Aforo, alertas y pases desde el panel del operador
LineasOficialesTest	19	90	Datos de las 8 líneas oficiales, orden y horarios
ImportarKmlTest	13	27	Lectura de archivos KML y KMZ
TrazoRutaTest	11	33	Editor de trazo y distancias
ReportesTest	16	49	Reportes, filtros, CSV y consultas documentadas
Otras (ejemplo)	4	11	Pruebas base del framework
Total	**146**	**386**	**100 % aprobadas en 6.3 segundos**

Indicador de calidad	Valor
Porcentaje de pruebas aprobadas	100 % (146 de 146)
Aserciones por prueba	2.6
Relación líneas de prueba / líneas de lógica de la aplicación	0.38
Ejecución automática	En cada cambio, con GitHub Actions (PostgreSQL 16 + Pint + Pest)
Aislamiento de las pruebas	Esquema “pruebas” separado; no tocan los datos reales

12.5. Métricas de calidad: defectos

Defectos encontrados y corregidos durante el desarrollo:

No.	Defecto	Detectado en	Estado
1	El procedimiento sp_cambiar_parqueo_bus fallaba al cambiar el parqueo de un bus	Pruebas automatizadas	Corregido
2	El trigger de seguridad no impedía eliminar al último guardia de un acceso	Pruebas automatizadas	Corregido
3	Fechas y horas registradas en UTC en lugar de la hora de Guatemala	Revisión funcional	Corregido
4	Identificador HTML duplicado en el selector de rol	Revisión funcional	Corregido
5	Las tarjetas de prueba no recibían los 5 viajes gratis	Pruebas automatizadas	Corregido
6	Texto ilegible sobre los colores claros de algunas líneas	Revisión del usuario	Corregido
7	Líneas en orden alfabético (L1, L12, L13, L18, L2) en lugar de orden natural	Revisión del usuario	Corregido
8	El trazo de las rutas no seguía las calles	Revisión del usuario	Corregido
9	Las pruebas usaban SQLite y podían borrar datos de desarrollo	Revisión técnica	Corregido

Indicador	Valor	Cálculo
Densidad de defectos	1.70 defectos por KLOC	9 defectos ÷ 5.294 KLOC
Defectos cerrados	100 %	9 de 9
Defectos detectados por pruebas automatizadas	33 %	3 de 9

12.6. Métricas de proceso y avance

Indicador	Estimado	Real / actual
Puntos de función ajustados	418.46 (alcance total)	390.40 implementados (93.3 %)
Funciones (transacciones)	41	36 implementadas (87.8 %)
Requerimientos funcionales	22	16 completos y 6 parciales (72.7 % completos)
Líneas de código	20,923 (PFA × 50)	5,294 de producción
Productividad en líneas por punto de función	50 LOC/PF	13.6 LOC/PF (5,294 ÷ 390.40)

La diferencia entre las líneas estimadas y las reales se debe a que Laravel resuelve gran parte del trabajo (ORM, validación, autenticación y migraciones). Para futuras estimaciones con este framework conviene usar un valor cercano a 15 líneas por punto de función.

12.7. Métricas pendientes

Métrica	Herramienta	Meta	Estado
Cobertura de código	Pest con PCOV en GitHub Actions	80 % o más en servicios y modelos	Pendiente
Errores, vulnerabilidades y code smells	SonarQube Cloud (gratis hasta 50,000 LOC privadas)	0 críticos; calificación A	Pendiente
Código duplicado	SonarQube Cloud	Menos de 3 %	Pendiente
Deuda técnica	SonarQube Cloud	Menos de 5 % (calificación A)	Pendiente
Tiempo de respuesta	Prueba de carga con k6 o JMeter sobre Render	95 % de solicitudes en menos de 2 s con 50 usuarios	Pendiente (tras el despliegue)
Disponibilidad	Monitor externo (UptimeRobot)	99 % o más en el horario de servicio	Pendiente (tras el despliegue)
Satisfacción del usuario	Cuestionario SUS a operadores	70 puntos o más	Pendiente
Tiempo medio de corrección (MTTR)	Issues de GitHub	Menos de 48 horas	Pendiente